A project report on

TWINSHIELD-FUSION: A DIGITAL TWIN-ASSISTED ADAPTIVE MULTI-ECU RESIDUAL FUSION FRAMEWORK FOR INTRUSION DETECTION ON ELECTRIC VEHICLE CONTROLLER AREA NETWORKS

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering

by

RAMYA NAKSHATHRA R (23BCE1611)



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

Octomber,2026

DECLARATION

I hereby declare that the thesis entitled "**TwinShield-Fusion: A Digital Twin-Assisted Adaptive Multi-ECU Residual Fusion Framework for Intrusion Detection on Electric Vehicle Controller Area Networks**" submitted by **RAMYA NAKSHATHRA R (23BCE1611)**, for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of Dr. Anubha Pearline.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date: **Signature of the Candidate**



School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled **"TwinShield-Fusion: A Digital Twin-Assisted Adaptive Multi-ECU Residual Fusion Framework for Intrusion Detection on Electric Vehicle Controller Area Networks"** is prepared and submitted by **RAMYA NAKSHATHRA R (REG. NO.)** to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:
Name: Dr. Anubha Pearline
Date:

Signature of the Examiner
Name:
Date:

Signature of the Examiner
Name:
Date:

Approved by the Head of Department,
B.Tech. Computer Science and Engineering
Name: HoD Name
Date:

(Seal of SCHOOL OF COMPUTER SCIENCE AND ENGINEERING)

ABSTRACT

An electric vehicle carries out braking, steering and propulsion as digital commands exchanged between dozens of electronic control units over the Controller Area Network (CAN). Drive-by-wire platforms place safety-critical actuation on that bus, yet a CAN frame carries no sender identity, no message authentication code and no encryption, so any node that reaches the bus can issue a command indistinguishable from a genuine one. Replacing the protocol would mean replacing every control unit on every vehicle already on the road. The practical defence is therefore an intrusion detection system: a passive monitor that observes bus traffic and raises an alarm when it departs from expected behaviour.

Learning-based intrusion detectors for CAN are almost always trained and evaluated on a fixed catalogue of attacks, and are then reported with closed-set metrics that quietly assume every attack seen at test time was also present during training. A vehicle in service meets attacks that were in nobody's catalogue, and a supervised classifier has no output neuron for them.

This project presents TwinShield-Fusion, an intrusion detection framework that pairs a conventional supervised detector with a second decision channel that never sees an attack during fitting. A gated recurrent unit digital twin is trained on normal traffic only and predicts the next payload on each arbitration identifier. The residual between prediction and observation is standardised per identifier and per payload byte, and an alarm threshold is calibrated on held-out normal frames at a target false-positive rate. Residuals are additionally pooled into per-ECU streams and combined by additive attention before being concatenated with a CNN-BiLSTM embedding of the raw context window.

The system is evaluated on GEM-CAN, 143,444 real frames captured from a GEM e6 autonomous electric vehicle on a closed track. Frames are split block-wise into 100,352 training, 21,504 validation and 21,588 test frames, with whole blocks of 256 consecutive frames assigned to one split at a time so that near-duplicate traffic cannot appear on both sides of the division. On the conventional closed-set benchmark TwinShield-Fusion reaches [ACC] per cent accuracy and 0.9979 macro-F1, and all four architectures compared fall within 0.0031 macro-F1 of one another, so that benchmark does not discriminate between them. Under a leave-one-attack-out protocol, in which

one attack class is removed from the label space, from every training context window, from the digital twin's per-ECU history and from the gate's calibration set, the same four models separate sharply: the CNN, BiLSTM and CNN-BiLSTM baselines flag 0.84%, 0.00% and 25.21% of the 119 held-out frames of the quarantined class, while TwinShield-Fusion flags 100.00% at a false-positive rate of 2.26% on normal traffic.

The limits of the mechanism are reported with the same prominence as its successes. Brake tampering raises the twin's mean residual from 2.58 to 23.02 bytes on the attacked identifier and is detected; steering tampering leaves it at 5.28 bytes against 6.39 on normal traffic, below the normal level, and cannot be detected by a score defined as a positive deviation. Detection therefore does not require prior knowledge of an attack, but it does require the attack to be anomalous in the payload. These are different conditions, and the project quantifies that boundary rather than asserting past it.

Keywords: *Controller Area Network, electric vehicle security, intrusion detection, digital twin, GRU, CNN-BiLSTM, anomaly detection*

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to Dr. [Guide Name], [Designation], School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for the constant guidance, continual encouragement and understanding; more than all, for teaching me patience in my endeavor. My association with my guide is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and expert in the field of machine learning and network security.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan our Honorable Chancellor, Mr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G V Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr. T. Thyagarajan Pro-Vice Chancellor, VIT Chennai and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.

Special mention to Dr. Viswanathan V, Dean, Dr. Nithyanandam P, Associate Dean, Dr. Suganya G, Associate Dean, Dr. Sweetlin Hemalatha C, Associate Dean, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai for spending their valuable time and efforts in sharing their knowledge and for helping us in every aspect.

In jubilant state, I express ingeniously my whole-hearted thanks to HoD Name, Head of the Department, B.Tech. Computer Science and Engineering and the Project Coordinators for their valuable support and encouragement to take up and complete the thesis.

My sincere thanks to all the faculties and staffs at Vellore Institute of Technology, Chennai who helped me acquire the requisite knowledge. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai

Date: **RAMYA NAKSHATHRA R**

CONTENTS

CHAPTER 1 INTRODUCTION.....
1.1 BACKGROUND.....
1.2 MOTIVATION.....
1.3 PROBLEM STATEMENT.....
1.4 OBJECTIVES.....
1.5 SCOPE OF THE PROJECT.....
1.6 CONTRIBUTIONS AND NOVELTY.....
1.7 ORGANISATION OF THE REPORT.....
CHAPTER 2 LITERATURE SURVEY.....
2.1 INTRODUCTION.....
2.2 THE CAN PROTOCOL AND ITS SECURITY PROPERTIES.....
2.3 DEMONSTRATED ATTACKS ON PRODUCTION VEHICLES.....
2.4 RULE AND SPECIFICATION BASED DETECTION.....
2.5 SUPERVISED DEEP LEARNING FOR CAN IDS.....
2.6 UNSUPERVISED AND RECONSTRUCTION-BASED DETECTION.....
2.7 PHYSICAL FINGERPRINTING OF ECUs.....
2.8 DIGITAL TWINS FOR SECURITY MONITORING.....
2.9 OPEN-SET RECOGNITION.....
2.10 EXPLAINABILITY IN INTRUSION DETECTION.....
2.11 COMPARATIVE SUMMARY.....
2.12 RESEARCH GAPS.....
2.13 SUMMARY.....
CHAPTER 3 REQUIREMENT ANALYSIS AND SPECIFICATION.....
3.1 INTRODUCTION.....
3.2 FEASIBILITY STUDY.....
3.2.1 TECHNICAL FEASIBILITY.....
3.2.2 ECONOMIC FEASIBILITY.....
3.2.3 OPERATIONAL FEASIBILITY.....
3.3 FUNCTIONAL REQUIREMENTS.....
3.4 NON-FUNCTIONAL REQUIREMENTS.....
3.5 HARDWARE REQUIREMENTS.....
3.6 SOFTWARE REQUIREMENTS.....

3.7 DATASET AND EXTERNAL ASSETS.....
3.8 CONSTRAINTS AND ASSUMPTIONS.....
3.9 SUMMARY.....
CHAPTER 4 SYSTEM DESIGN.....
4.1 INTRODUCTION.....
4.2 SYSTEM ARCHITECTURE.....
4.3 END-TO-END PIPELINE.....
4.4 USE CASE DESIGN.....
4.5 DATA FLOW DESIGN.....
4.5.1 CONTEXT DIAGRAM (LEVEL 0).....
4.5.2 LEVEL-1 DATA FLOW DIAGRAM.....
4.6 SEQUENCE OF A DETECTION REQUEST.....
4.7 MODULE STRUCTURE.....
4.8 ARTIFACT DESIGN.....
4.9 DEPLOYMENT DESIGN.....
4.10 SUMMARY.....
CHAPTER 5 METHODOLOGY AND IMPLEMENTATION.....
5.1 INTRODUCTION.....
5.2 DATASET: GEM-CAN.....
5.3 FRAME-LEVEL FEATURE ENGINEERING.....
5.4 CONTEXT WINDOWS AND LEAKAGE-CONTROLLED SPLITTING.....
5.5 THE GRU DIGITAL TWIN.....
5.6 RESIDUAL COMPUTATION.....
5.7 ADAPTIVE MULTI-ECU RESIDUAL FUSION.....
5.8 BASELINE DETECTORS.....
5.9 THE TWINSHIELD-FUSION DETECTOR.....
5.10 THE RESIDUAL ANOMALY GATE.....
5.11 THE LEAVE-ONE-ATTACK-OUT PROTOCOL.....
5.12 EXPLAINABILITY WITH SHAP.....
5.13 THE APPLICATION.....
5.14 SUMMARY.....
CHAPTER 6 RESULTS AND DISCUSSION.....
6.1 INTRODUCTION.....
6.2 EVALUATION METRICS.....

6.3 DATASET AUDIT AND THE ARBITRATION-ID SHORTCUT.....
6.4 DIGITAL TWIN PREDICTION QUALITY.....
6.5 CLOSED-SET COMPARISON.....
6.6 ABLATION STUDY.....
6.7 DETECTION OF A QUARANTINED ATTACK.....
6.8 WHERE THE RESIDUAL WORKS, AND WHERE IT DOES NOT.....
6.9 CLOSED-SET AND OPEN-SET RESULTS COMPARED.....
6.10 THE COST OF THE SECOND CHANNEL.....
6.11 RUNTIME.....
6.12 USER INTERFACE.....
6.14 DISCUSSION.....
6.15 LIMITATIONS.....
6.16 SUMMARY.....
CHAPTER 7 TESTING AND VALIDATION.....
7.1 INTRODUCTION.....
7.2 TESTING STRATEGY.....
7.3 TEST CASES.....
7.4 DEFECTS FOUND AND FIXED.....
7.5 SUMMARY.....
CHAPTER 8 CONCLUSION AND FUTURE WORK.....
8.1 CONCLUSION.....
8.2 FUTURE WORK.....
8.3 SUMMARY.....
REFERENCES.....
APPENDIX 1 ALGORITHMS.....
A1.1 DIGITAL TWIN TRAINING.....
A1.2 RESIDUAL ANOMALY GATE: FITTING AND CALIBRATION.....
A1.3 SCORING A FRAME.....
A1.4 LEAVE-ONE-ATTACK-OUT QUARANTINE.....
APPENDIX 2 INSTALLATION AND RUNNING.....
A2.1 ENVIRONMENT SETUP.....
A2.2 RUNNING THE SYSTEM.....
A2.3 COMMAND-LINE OPTIONS.....
APPENDIX 3 ARTIFACT AND PARAMETER REFERENCE.....

A3.1 MODEL HYPERPARAMETERS.....
A3.2 SPLIT COMPOSITION.....

LIST OF FIGURES

Figure 1.1: Signature-based detection compared with the TwinShield-Fusion approach
Figure 2.1: Taxonomy of approaches to CAN intrusion detection.....
Figure 4.2: Layered view of the same system.....
Figure 4.3: End-to-end processing pipeline: seven stages from raw frames to alarm.....
Figure 4.4: Use case diagram.....
Figure 4.5: Level-0 data flow diagram (context diagram).....
Figure 4.6: Level-1 data flow diagram.....
Figure 4.7: Sequence diagram of one replay tick.....
Figure 5.1: The GRU digital twin: predicting the next payload on an arbitration identifier.....
Figure 5.2: Adaptive multi-ECU residual fusion by additive attention.....
Figure 5.3: The two-view residual anomaly gate.....
Figure 5.4: Leave-one-attack-out quarantine applied at four separate stages.....
Figure 6.1: Closed-set macro-F1 for all five configurations.....
Figure 6.2: Ablation ladder with and without the arbitration identifier.....
Figure 6.3: Detection of a quarantined attack class against the false-positive cost.....
Figure 6.4: Mean digital-twin residual on the two tampering identifiers.....
Figure 6.5: The same four models under two evaluation protocols.....
Figure 6.6: Live monitor on the known-attack control case.....
Figure 6.7: Live monitor on the quarantined attack case.....
Figure 6.8: Results page: unknown-attack generalisation.....

LIST OF TABLES

Table 2.1: Comparison of detection families against unknown-attack requirements……
Table 3.1: Functional requirements.....
Table 3.2: Non-functional requirements.....
Table 3.3: Hardware requirements.....
Table 3.4: Software requirements.....
Table 4.1: Module structure.....
Table 4.2: Artifacts produced by training.....
Table 5.1: Composition of the GEM-CAN capture.....
Table 6.1: Dataset audit findings.....
Table 6.2: Closed-set macro-F1 on the shared held-out test split.....
Table 6.3: Ablation on the CNN-BiLSTM backbone (macro-F1).....
Table 6.4: Leave-one-attack-out: brake tampering quarantined from every training stage.....
Table 7.1: Levels of testing.....
Table 7.2: Representative test cases.....
Table 7.3: Significant defects found during development.....
Table A2.1: Principal command-line options.....
Table A3.1: Model hyperparameters.....
Table A3.2: Class support in each split.....

LIST OF ABBREVIATIONS

API	Application Programming Interface
AUC	Area Under the Curve
BiLSTM	Bidirectional Long Short-Term Memory
CAN	Controller Area Network
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DLC	Data Length Code
DoS	Denial of Service
ECU	Electronic Control Unit
EV	Electric Vehicle
F1	Harmonic mean of precision and recall
FPR	False-Positive Rate
GAN	Generative Adversarial Network
GEM	Global Electric Motorcars
GRU	Gated Recurrent Unit
ID	Arbitration Identifier
IDS	Intrusion Detection System
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
ML	Machine Learning
MSE	Mean Squared Error
OBD	On-Board Diagnostics
ReLU	Rectified Linear Unit
RMSE	Root Mean Squared Error
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
TPR	True-Positive Rate

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

The Controller Area Network has been the backbone of in-vehicle communication for more than three decades. It was standardised as ISO 11898 and designed for a closed environment in which every node on the bus was assumed to be trustworthy, because reaching the bus required physical access to the vehicle's wiring. Under that assumption the protocol's design is elegant: frames carry an 11-bit or 29-bit arbitration identifier, up to eight payload bytes and a short checksum, and bus contention is resolved by the numeric value of the identifier alone, so the message with the lowest identifier always wins.

The assumption no longer holds. Modern vehicles expose infotainment systems, telematics units, Bluetooth and cellular interfaces, tyre-pressure sensors and on-board diagnostics ports, and each of these has at some point been shown to provide a path onto the internal network. What makes this serious is that the protocol offers no defence once that path exists. A CAN frame carries no sender identity, no message authentication code and no encryption. A receiving ECU cannot distinguish a brake command issued by the genuine brake controller from an identical frame injected by an attacker, because at the protocol level the two are the same sequence of bits.

The electric vehicle has made this inheritance more consequential. An internal combustion vehicle retains mechanical linkages between the driver and the wheels; an electric platform, and an autonomous one in particular, replaces them with drive-by-wire actuation in which braking, steering and propulsion are issued as CAN frames. Battery management, regenerative braking and motor torque control add further high-rate traffic to the same bus. The result is that more safety-critical function is expressed as bus messages, on a vehicle whose connectivity gives an attacker more ways to reach that bus.

The consequences have been demonstrated repeatedly on production vehicles. Koscher et al. (2010) showed that a laptop connected to the diagnostics port could disable brakes, stop the engine and manipulate the instrument cluster of an unmodified passenger car. Checkoway et al. (2011) extended the result by establishing that the same effects could

be reached remotely, through the vehicle's cellular and Bluetooth interfaces, without any physical contact. Miller and Valasek (2015) published the best-known demonstration, remotely steering a Jeep Cherokee off a public highway, which led to the recall of 1.4 million vehicles.

Replacing CAN with an authenticated protocol would require replacing every ECU on every vehicle, which is not economically realistic for a fleet already on the road. The practical response adopted by both industry and the research community has therefore been to add an intrusion detection system. An IDS is a passive monitor: it reads every frame on the bus, forms a judgement about whether traffic looks the way it should, and raises an alarm when it does not. It changes no message and blocks nothing, which is what makes it deployable on an existing vehicle, and it is the only defence that can be retrofitted without touching the protocol. Intrusion detection for CAN divides into two families: signature-based detectors, which look for patterns known to be malicious, and anomaly-based detectors, which model what is normal and report departures from it. This project is concerned with the question of what “should” means in that judgement, and in particular with what happens when an attack does not resemble anything the monitor was taught to recognise.

Challenges. Four properties of the problem make intrusion detection on a vehicle bus harder than it first appears.

The traffic is extremely repetitive. ECUs transmit fixed status payloads at fixed intervals, so a large share of frames is byte-identical to another frame elsewhere in the capture. A model can score well by memorising the repetition rather than learning the behaviour, and a careless train-test split will flatter it further by placing near-duplicate frames on both sides.

The classes are severely imbalanced. A flooding attack produces tens of thousands of frames, while a tampering attack produces a few hundred. Accuracy is dominated by the majority classes and reports almost nothing about the rare ones, which are the attacks that matter.

Some attacks are trivially separable and some are not. A denial-of-service flood announces itself through an arbitration identifier no legitimate ECU uses, so a lookup table detects it. Tampering attacks forge payloads on identifiers that carry genuine

traffic throughout the capture, so the identifier carries no information and the decision must come from the payload or the timing.

The attack set is open. A detector trained on a catalogue of attacks must still behave sensibly when it meets one that is absent from that catalogue, and a closed-set evaluation never asks it to.

1.2 MOTIVATION

The motivation for TwinShield-Fusion comes from three observations made while surveying the existing literature at the start of the project.

First, the reported numbers in CAN intrusion detection are extraordinarily high. Accuracies above 99% are routine across a wide range of architectures, and papers regularly report figures that would, in any other domain, suggest the problem is solved. Those numbers are also, in a specific and important sense, uninformative. They are closed-set results: the model is trained on a catalogue of attacks, tested on the same catalogue, and scored on how often it names the correct member of it. Nothing in that protocol asks what happens when an attack arrives that was not in the catalogue, which is precisely the situation a deployed detector will eventually be in.

Second, the failure mode is structural rather than incidental. A supervised classifier with a softmax over k trained classes can only ever emit one of those k labels. An attack outside the set must be assigned to whichever of them fits least badly, and for traffic that does not resemble a flood, the least bad fit is almost always the normal class. Adding convolutional or recurrent capacity does not change this, because the missing ingredient is not capacity. It is a reference for what the bus ought to be doing, independent of what attacks anyone has catalogued.

Third, such a reference is obtainable. Each ECU on a vehicle repeats a fairly predictable pattern of messages, because it is reporting the state of a physical subsystem that evolves smoothly. A model fitted only to clean traffic can learn that pattern and predict what the next message on a given identifier ought to contain. The gap between prediction and observation is a measure of deviation from learnt-normal behaviour, and crucially it is defined for any frame whatsoever, including a frame belonging to an attack that nobody has ever labelled.

A fourth motivation emerged during development and shaped the project's presentation as much as its design. Several intermediate versions produced outputs that looked convincing but could not be justified. The most instructive was a closed-set comparison in which every architecture scored above 0.99 macro-F1; presented without comment, that table appears to show a strong result, when in fact it shows a saturated benchmark that has stopped measuring anything. A recurring principle of this work is therefore that every number reported must be traceable to a measurement, and that where a result is negative or a benchmark is uninformative, the report must say so plainly.

1.3 PROBLEM STATEMENT

Given a stream of CAN frames captured from a vehicle bus, where each frame carries an arbitration identifier, a data length code, up to eight payload bytes and a timestamp, the problem is to:

1. detect that an attack is in progress, reporting a decision for each individual frame;
2. name the attack class when it belongs to a family the system has been trained on;
3. raise an alarm on an attack that was absent from every stage of training, where no class label can be assigned;
4. report the cost of that capability as a false-positive rate on genuine traffic; and
5. explain which features and which ECU streams drove each decision,

while being trained and evaluated under a protocol that controls for data leakage between splits, and while making no claim about attack classes that the dataset does not contain.

1.4 OBJECTIVES

The specific objectives of the project are:

1. To audit the GEM-CAN dataset for shortcuts and leakage before any model is trained, and to design a splitting strategy that prevents near-duplicate frames from appearing in both training and test sets.

2. To train a GRU digital twin on normal CAN traffic alone, such that it predicts the next payload on a given arbitration identifier from the recent history of that identifier, and to quantify its prediction quality on held-out normal frames.
3. To derive a per-byte residual from the twin and to pool residuals into per-ECU streams combined by additive attention, so that evidence concentrates on whichever ECU is being attacked.
4. To implement three supervised baselines (CNN, BiLSTM and CNN-BiLSTM) and the proposed TwinShield-Fusion detector, and to compare them on one shared, leakage-controlled test split.
5. To design a residual anomaly gate whose statistics and alarm threshold are estimated from normal traffic alone, so that no attack signature can be memorised by it.
6. To specify and execute a leave-one-attack-out protocol that removes an attack class from the label space, the training context windows, the digital twin's per-ECU history and the gate's calibration set, and to compare all four models under it.
7. To measure and report the false-positive cost of the additional detection channel, rather than reporting detection rates in isolation.
8. To attribute decisions to individual features and ECU streams using SHAP, and to deliver the whole system through an interactive application in which every displayed number is produced by a real forward pass.

1.5 SCOPE OF THE PROJECT

The scope of this project is deliberately bounded so that each component can be validated rather than merely demonstrated.

Applications of the work. The system is built and evaluated as a passive bus monitor for an electric, drive-by-wire vehicle, and this is the setting in which it applies: an in-vehicle intrusion detection unit on a platform where braking and steering are issued as CAN messages. It operates on the GEM-CAN capture, comprising 143,444 real frames recorded from a GEM e6 autonomous electric vehicle on a closed track with a 250 kbit/s bus. Four classes are supported, being exactly those the dataset contains: normal traffic, denial-of-service flooding, brake tampering and steering tampering. Detection is

performed at frame level using a 32-frame context window. Every model is trained twice, once with the arbitration identifier available as an input and once with it removed, so that the identifier shortcut identified in the audit can be eliminated rather than merely acknowledged. The leave-one-attack-out experiment is run with brake tampering as the quarantined class. Beyond the vehicle itself, the same construction applies wherever a fleet operator or a security tester needs a detector that reports an attack it was never trained on, and the interactive application built in Chapter 5 is intended for exactly that kind of inspection.

Boundaries of the study. The project stops short of real-time deployment on embedded automotive hardware, and makes no claim about line-rate operation on a vehicle ECU. Attack prevention, message authentication and key management lie outside it, since all three would require protocol changes. Validation is confined to a single capture, so claims across multiple vehicles, drive cycles or attack families are left for future work. Replay attacks, in which a genuine past message is retransmitted at the wrong moment, are discussed as a known limitation; the dataset contains none, so they are not evaluated.

1.6 CONTRIBUTIONS AND NOVELTY

The key contributions of this work, relative to typical learning-based CAN intrusion detectors, are summarised below and elaborated in later chapters.

- **A quarantine protocol that is actually airtight.** Removing an attack class from the label set is not sufficient, because a frame labelled normal whose context window contains injected frames has still shown the detector what the attack looks like. The protocol additionally drops contaminated windows, refits the digital twin on histories free of the class, and calibrates the gate on normal frames only.
- **A two-view residual anomaly gate.** The digital-twin residual is standardised per arbitration identifier and per payload byte for the frame view, and per ECU stream across the context window for the sustained view. This per-stream standardisation, rather than a single global threshold, is what makes one bus-wide operating point usable across ECUs with very different baseline predictability.

- **Evidence that the closed-set benchmark is saturated.** On this dataset the four architectures compared span 0.0031 macro-F1, which is reported as a property of the benchmark rather than dressed up as a win for the proposed method.
- **Attribution of the result to the correct component.** On the quarantined class the supervised head of the proposed model contributes none of the detection and the anomaly gate contributes all of it. The claim made is that the residual generalises, not that the architecture does.
- **A measured negative result, reported with equal prominence.** Held out in turn, steering tampering cannot be detected by the gate, because its forged payloads fall inside the range the twin already accepts for that identifier.

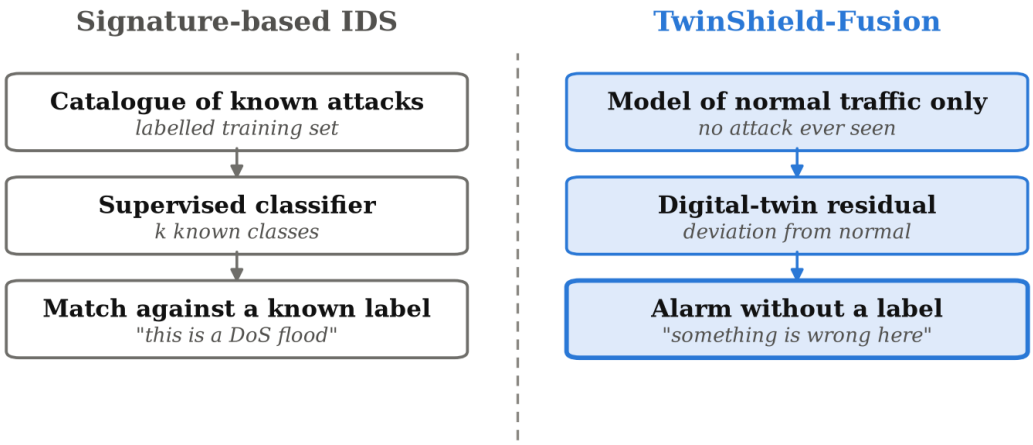


Figure 1.1: Signature-based detection compared with the TwinShield-Fusion approach

1.7 ORGANISATION OF THE REPORT

The remainder of this report is organised as follows. Chapter 2 surveys the literature on the CAN protocol and its weaknesses, demonstrated attacks, specification-based and supervised detection, unsupervised and reconstruction-based methods, ECU fingerprinting, digital twins and open-set recognition, and identifies the gaps this project addresses. Chapter 3 states the functional and non-functional requirements together with the hardware, software and dataset environment. Chapter 4 presents the system design: architecture, pipeline, use cases, data flow, sequence of operations, module structure and deployment. Chapter 5 describes the methodology and implementation of each component in detail. Chapter 6 presents and discusses the experimental results. Chapter 7 describes the testing strategy and the defects found and fixed during development. Chapter 8 concludes and outlines future work. References

and appendices containing algorithm listings, setup instructions and an artifact reference follow.

CHAPTER 2

LITERATURE SURVEY

2.1 INTRODUCTION

This chapter reviews the work on which this project builds. Section 2.2 describes the CAN protocol and the properties that make it attackable. Section 2.3 summarises the demonstrated attacks that motivate detection research. Sections 2.4 to 2.8 review the main families of detection method: rule and specification based, supervised learning, unsupervised reconstruction, physical fingerprinting and digital twins. Section 2.9 introduces open-set recognition, which frames the problem this project addresses. Section 2.10 covers explainability. Section 2.11 compares the families against the requirements of unknown-attack detection, Section 2.12 states the gaps, and Section 2.13 summarises.

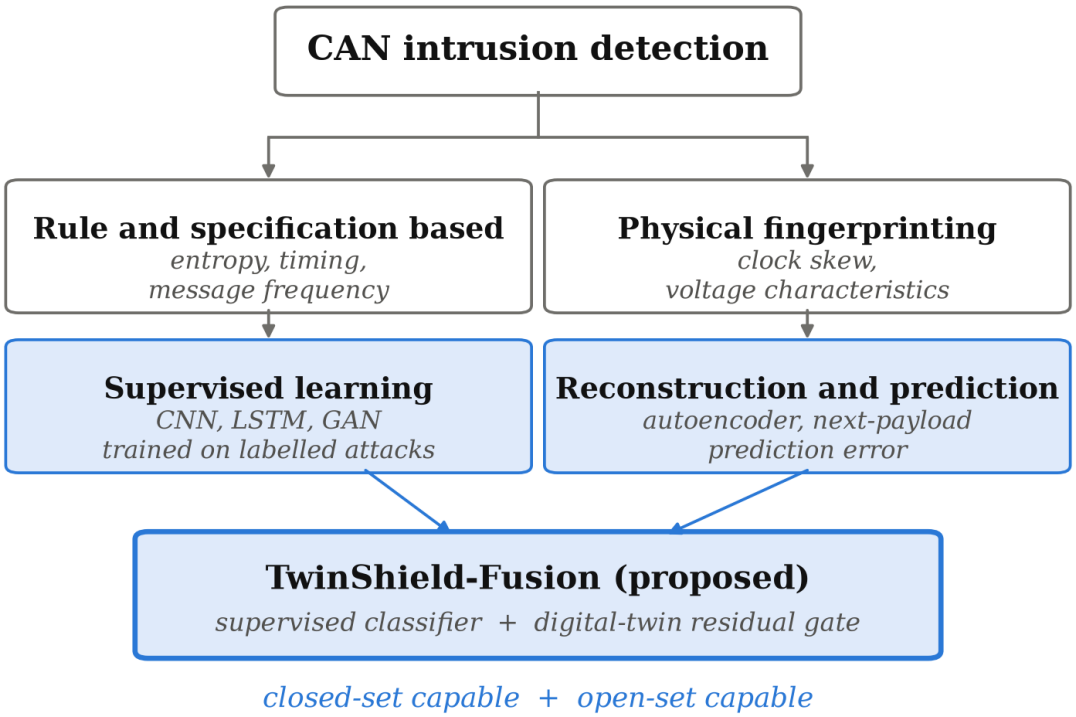


Figure 2.1: Taxonomy of approaches to CAN intrusion detection

2.2 THE CAN PROTOCOL AND ITS SECURITY PROPERTIES

CAN is a multi-master broadcast serial bus standardised as ISO 11898-1 (2015). Every node receives every frame and decides for itself whether the frame concerns it, based on

the arbitration identifier. When two nodes transmit at once, arbitration is resolved bit by bit: a dominant zero overwrites a recessive one, so the frame with the numerically lower identifier wins and the loser retries. This is efficient and fully deterministic, which is why the protocol has survived so long in safety-critical applications.

Three properties of this design create the security problem. There is no sender authentication, so a frame carries no evidence of which node transmitted it. There is no encryption, so payloads are plaintext and trivially replayable. And priority is determined entirely by the identifier value, so a node that transmits identifier zero repeatedly can monopolise the bus and starve every legitimate message. These are not defects in an implementation that could be patched; they are the protocol.

2.3 DEMONSTRATED ATTACKS ON PRODUCTION VEHICLES

Koscher et al. (2010) performed the first systematic security analysis of a modern automobile, reverse-engineering ECU firmware and demonstrating control of brakes, engine and instrumentation from the diagnostics port. Checkoway et al. (2011) classified the full remote attack surface, showing that the media player, Bluetooth stack and cellular telematics unit all provided paths onto the internal buses. Miller and Valasek (2015) combined such a path with CAN injection to take remote control of a moving vehicle. Collectively this work established that in-vehicle network security cannot rely on physical isolation, and motivated the intrusion detection literature reviewed below.

2.4 RULE AND SPECIFICATION BASED DETECTION

The earliest detectors exploited the regularity of CAN traffic without any learning. Because most ECUs transmit at fixed intervals, injected frames disturb the message frequency of their identifier, and simple timing or frequency monitors catch high-rate injection reliably. Müter and Asaj (2011) proposed an entropy-based approach, measuring the information content of the message stream and flagging deviations from its normal level.

These methods are cheap, interpretable and need no labelled attack data, which makes them attractive for embedded deployment. Their weakness is that they are blind to attacks that preserve traffic statistics. An attacker who injects a small number of forged frames at the normal rate of the identifier being spoofed changes neither frequency nor

entropy appreciably, and the payload content, where the attack actually lives, is never examined.

2.5 SUPERVISED DEEP LEARNING FOR CAN IDS

The dominant line of recent work applies supervised deep learning to labelled attack datasets. Kang and Kang (2016) trained a deep neural network on CAN payloads and reported detection rates above 97%. Song et al. (2020) arranged consecutive frames into a two-dimensional grid, treating the bus as an image, and applied a reduced Inception-ResNet architecture. Seo et al. (2018) addressed the scarcity of attack samples with GIDS, using a generative adversarial network to synthesise additional attack traffic for training. Lokman et al. (2019) survey the field more broadly.

These methods differ considerably in architecture but share an assumption that is rarely stated: the attack classes present at test time are the attack classes present at training time. Their evaluation protocols inherit that assumption, and the resulting metrics therefore measure interpolation within a catalogue rather than response to anything outside it. This project does not dispute their closed-set results; it argues that those results answer a different question from the one a deployed detector faces.

2.6 UNSUPERVISED AND RECONSTRUCTION-BASED DETECTION

A second line of work discards the attack labels entirely and models normal traffic. Taylor et al. (2016) train an LSTM to predict the next payload on an identifier and threshold the prediction error, which is the closest antecedent of the mechanism used in this project. Hanselmann et al. (2020) generalise the idea in CANet, an autoencoder over multiple signals trained on attack-free traffic, with reconstruction error as the anomaly score. Chandola et al. (2009) give the general framing of anomaly detection on which both rest.

The contribution of the present work relative to these is not the idea of thresholding a prediction error, which is well established. It is the evaluation. Those studies report detection of attacks absent from training, but they do not place the unsupervised channel alongside supervised baselines under an identical, independently verified quarantine. Without that comparison it is impossible to separate how much of the detection is attributable to the model and how much to the fact that it did not need a label.

2.7 PHYSICAL FINGERPRINTING OF ECUs

Cho and Shin (2016) take an orthogonal route, fingerprinting ECUs by the clock skew of their periodic transmissions. Because each microcontroller's oscillator drifts slightly differently, the intervals between successive frames on an identifier carry a signature of the physical device producing them, and a frame transmitted by a different device can be detected even if its content is perfect. Related work uses voltage-level characteristics for the same purpose.

Fingerprinting identifies the transmitter rather than the content, which makes it complementary to payload modelling rather than competing with it. It is not applicable here, because the GEM-CAN capture provides neither the timing resolution nor the physical-layer measurements such methods require.

2.8 DIGITAL TWINS FOR SECURITY MONITORING

A digital twin is a model maintained in correspondence with a physical system, used to predict or interpret its behaviour. Tao et al. (2019) survey its industrial applications. In security monitoring the appeal is that the twin encodes what the system should be doing, so a discrepancy between twin and reality is evidence of interference, regardless of how that interference was produced.

Most security applications of digital twins are in industrial control systems, where process physics tightly constrains plausible sensor readings. CAN payloads on a passenger vehicle are less tightly constrained, and some bytes, such as rolling counters, are not predictable in principle. The principle nonetheless survives: an ECU that repeats a predictable message pattern can be modelled, and departures from that model are evidence. Quantifying how far that survives on real vehicle traffic is part of what this project measures.

2.9 OPEN-SET RECOGNITION

The problem addressed by this project has a name in the machine learning literature. Geng et al. (2021) survey open-set recognition, the setting in which test data may contain classes absent from training and the classifier must be able to reject rather than assign. The standard observation is that a softmax classifier is closed by construction:

its outputs are a probability distribution over the training classes and sum to one regardless of how unlike any of them the input is.

Applying this framing to CAN intrusion detection clarifies what the baselines in this project can and cannot do, and why an additional channel rather than a better classifier is the appropriate response. It also sets the evaluation: a closed-set metric such as accuracy or macro-F1 is not defined across models with different label spaces, so the comparison must be made on a question that is well posed in any label space, namely whether an alarm was raised.

2.10 EXPLAINABILITY IN INTRUSION DETECTION

An intrusion detection system that cannot justify its alerts is difficult to trust and difficult to debug. Lundberg and Lee (2017) introduced SHAP, which attributes a model's output to its input features using a game-theoretic allocation with desirable consistency properties. This project applies SHAP to the trained detector over both the raw context window and the per-ECU residual streams, so that an alert can be traced to specific payload bytes, timing features and ECUs.

2.11 COMPARATIVE SUMMARY

Table 2.1 compares the families of approach against the requirements of unknown-attack detection on a vehicle bus.

Table 2.1: Comparison of detection families against unknown-attack requirements

Approach	Needs attack labels	Detects unseen attacks	Examines payload	Names the class
Rule / specification based	No	Partly	No	No
Entropy / frequency monitors	No	Partly	No	No
Supervised deep learning	Yes	No	Yes	Yes
GAN-augmented supervised	Yes	No	Yes	Yes
ECU fingerprinting	No	Yes	No	No
Reconstruction / prediction error	No	Yes	Yes	No
TwinShield-Fusion (proposed)	For known classes only	Yes	Yes	For known classes

No single family satisfies every requirement, and the table makes the design choice of this project explicit. A supervised classifier is retained because naming a known attack is genuinely useful to an operator, and an unsupervised residual channel is added alongside it because naming is exactly what cannot be done for an attack with no label.

2.12 RESEARCH GAPS

Four gaps emerge from the survey and define the work reported here.

1. Evaluation protocols are closed-set. The dominant metrics assume the test-time attack catalogue matches the training-time one, and therefore cannot distinguish a detector that generalises from one that has memorised a catalogue.
2. Quarantine, where attempted, is usually partial. Removing a class from the labels leaves it present in context windows and in the history used to fit any normal-traffic model, so the claim that a model has never seen an attack is rarely airtight.
3. Unsupervised channels are seldom compared head-to-head with supervised baselines under identical conditions, which makes it impossible to attribute an improvement to the mechanism rather than to the experimental setup.
4. Detection rates are frequently reported without the corresponding false-positive rate on genuine traffic, although an always-on alarm trivially achieves perfect detection.

2.13 SUMMARY

This chapter reviewed the CAN protocol's security properties, the attacks that exploit them and the main families of detection method. Supervised deep learning dominates the recent literature and reports excellent closed-set results, but is structurally unable to respond to an attack outside its label set. Reconstruction and prediction-error methods can respond to such attacks, but have not been evaluated against supervised baselines under a verified quarantine. The next chapter states the requirements of a system designed to close that gap.

CHAPTER 3
REQUIREMENT ANALYSIS AND SPECIFICATION

3.1 INTRODUCTION

This chapter states what the system must do and the environment in which it must do it. Section 3.2 assesses feasibility. Sections 3.3 and 3.4 give the functional and non-functional requirements. Sections 3.5 and 3.6 specify the hardware and software environment, Section 3.7 describes the dataset and external assets, and Section 3.8 records the constraints and assumptions under which the results in Chapter 6 should be read.

3.2 FEASIBILITY STUDY

3.2.1 TECHNICAL FEASIBILITY

Every component required is openly available. The GEM-CAN dataset is published on Zenodo under a CC-BY-NC-3.0 licence. PyTorch provides the GRU, convolutional and recurrent layers; scikit-learn provides the metrics; SHAP provides the attribution method; Streamlit provides the interface. All training and inference reported in this project were performed on CPU, so no specialised accelerator is required. The heaviest single operation is the full training run, which completes in under an hour on a conventional laptop.

3.2.2 ECONOMIC FEASIBILITY

The project incurs no licensing or infrastructure cost. Every library used is open source, the dataset is free for non-commercial academic use, and the computation runs on existing hardware. The only cost is development time.

3.2.3 OPERATIONAL FEASIBILITY

The system is operated through a browser interface with three pages and a small number of controls, so an evaluator with no knowledge of the implementation can run the comparison and read the results. Training is performed offline and the application loads saved checkpoints, so no page in the interface ever waits on a training run. A readiness check script verifies that every artifact is present and performs one real inference before a demonstration begins.

3.3 FUNCTIONAL REQUIREMENTS

Table 3.1: Functional requirements

ID	Requirement
FR-1	The system shall load the GEM-CAN capture and derive per-frame class labels from the published attack-type annotations.
FR-2	The system shall construct frame-level features comprising eight payload bytes, data length code and timing, in two variants: with and without the arbitration identifier.
FR-3	The system shall assign contiguous blocks of frames to training, validation and test splits such that no frame's context window crosses a split boundary.
FR-4	The system shall train a GRU digital twin on normal training frames only, predicting the next payload on a given arbitration identifier.
FR-5	The system shall compute a per-byte residual as the absolute difference between the observed and predicted payload.
FR-6	The system shall pool residuals into per-ECU streams over the context window and combine them with additive attention.
FR-7	The system shall train CNN, BiLSTM, CNN-BiLSTM and TwinShield-Fusion detectors on one shared split.
FR-8	The system shall fit an anomaly gate whose per-identifier and per-stream statistics are estimated on normal training frames and whose threshold is set on normal validation frames.
FR-9	The system shall support a leave-one-attack-out mode that removes a named attack class from the label space, from contaminated context windows, from the twin's per-ECU history and from the gate's calibration set.
FR-10	The system shall report, per model and per true class, the alarm rate on the held-out test split.
FR-11	The system shall compute SHAP attributions over both the context window and the per-ECU streams.
FR-12	The system shall replay recorded frames through all four detectors and display per-model verdicts, catch counts and false-alarm counts.
FR-13	The system shall display the anomaly score against its calibrated threshold for each replayed frame.
FR-14	The system shall provide a readiness check that verifies all artifacts and performs one real inference.

3.4 NON-FUNCTIONAL REQUIREMENTS

Table 3.2: Non-functional requirements

ID	Category	Requirement
NFR-1	Correctness	Every number displayed by the application shall be produced by a real forward pass or read from a file written by a training run. No value shall be hard-coded.
NFR-2	Reproducibility	All experiments shall run from a fixed random seed, so that a repeated run reproduces the reported figures.

ID	Category	Requirement
NFR-3	Performance	A replay tick shall complete inference for all four models within 100 ms so that the interface remains responsive.
NFR-4	Transparency	Where a benchmark is saturated or a result is negative, the application and the report shall state so explicitly rather than omit it.
NFR-5	Usability	An evaluator shall be able to run the full comparison without reading the source code.
NFR-6	Maintainability	Training, explanation and interface concerns shall be separated into distinct modules with a documented interface.
NFR-7	Portability	The system shall run on CPU on Windows, Linux and macOS without code changes.

3.5 HARDWARE REQUIREMENTS

Table 3.3: Hardware requirements

Component	Minimum	Used in this project
Processor	Quad-core x86-64, 2.0 GHz	Intel Core i5 class, 8 logical cores
Memory	8 GB RAM	16 GB RAM
Storage	2 GB free	SSD, approximately 1.5 GB used by artifacts
Graphics	Not required	Not used; all training and inference on CPU
Display	1366 x 768	1920 x 1080

3.6 SOFTWARE REQUIREMENTS

Table 3.4: Software requirements

Component	Version	Purpose
Python	3.11	Implementation language
PyTorch	2.2 or later	GRU, CNN, BiLSTM and attention layers; training loop
NumPy	1.26 to 2.2	Array computation, residuals, gate statistics
pandas	2.1 or later	Dataset loading and frame tables
scikit-learn	1.4 or later	Metrics, class weighting, stratified assignment
SHAP	0.45 or later	Gradient-based feature attribution
Streamlit	1.40 or later	Interactive application
Plotly	5.20 or later	Interactive charts in the application

3.7 DATASET AND EXTERNAL ASSETS

The project uses GEM-CAN, recorded from a GEM e6 autonomous electric vehicle driven on a closed track with a 250 kbit/s CAN bus. The dataset comprises two captures. The first records normal driving; the second records a session during which attacks were physically injected onto the bus, with per-frame attack-type labels. Together they contain 143,444 frames. The composition is given in Table 5.1.

Two properties of the published files required handling in the loader. The two attack CSV files ship with a corrupted first-column header, and the normal-driving capture ends with a trailing blank line. The loader normalises any first column that parses as a timestamp and discards rows without an arbitration identifier, so the files can be used exactly as published without manual editing. Arbitration identifiers and payload bytes are stored as hexadecimal strings and are converted on load.

No other external asset is required. No pre-trained weights are used; every model reported in this project was trained from random initialisation on this dataset.

3.8 CONSTRAINTS AND ASSUMPTIONS

The following constraints bound what the results in Chapter 6 can support, and are restated as limitations in Section 6.14.

- **Single capture.** All results come from one vehicle on one closed track. Performance across vehicles, drive cycles or bus configurations is not evidenced.
- **Three attack classes only.** The dataset contains denial of service, brake tampering and steering tampering. No claim is made about any other attack family, including replay, fuzzing or masquerade attacks.
- **Known identifiers.** The quarantined attack arrives on an identifier that also carries legitimate traffic, so the digital twin has a normal profile to compare against. An attack on an identifier never observed during training is a weaker case and is not tested.
- **Offline evaluation.** The system processes a recorded capture. It is not deployed on a live bus and makes no claim about line-rate operation on embedded hardware.

- **Residual redundancy.** CAN traffic is highly repetitive and the dataset contains a high proportion of byte-identical frames. The block-split strategy limits but does not eliminate the resulting inflation of frame-level metrics.

3.9 SUMMARY

This chapter specified fourteen functional and seven non-functional requirements, the hardware and software environment, the dataset and the constraints under which the system operates. The requirement that no displayed value may be hard-coded, and the requirement that negative results be stated explicitly, shaped the design presented in the next chapter as much as any functional requirement did.

CHAPTER 4
SYSTEM DESIGN

4.1 INTRODUCTION

This chapter presents the design of TwinShield-Fusion. Section 4.2 gives the system architecture and its layered view, Section 4.3 the end-to-end processing pipeline and Section 4.4 the use cases. Sections 4.5 and 4.6 give the data flow and the sequence of a detection request. Section 4.7 describes the module structure, Section 4.8 the artifacts produced by training, and Section 4.9 the deployment arrangement.

4.2 SYSTEM ARCHITECTURE

Figure 4.1 shows the architecture as a whole. A CAN frame is decomposed into its arbitration identifier, data length code, payload bytes and timestamp, and is then presented to two branches that run in parallel and share no fitting stage. The upper branch is the digital twin: it holds the recent payload history for the frame's own identifier, predicts the next payload with a GRU fitted on normal traffic alone, and forms the residual between prediction and observation. The lower branch is the supervised classifier: it gathers a 32-frame context window, passes it through the CNN-BiLSTM with adaptive multi-ECU residual fusion, and emits a posterior over the classes it was trained on. The residual anomaly gate standardises the residual per identifier and per payload byte and raises an alarm above a threshold calibrated on normal frames. The alarm decision fires if either channel fires: it reports a named class when the classifier recognises one, and an unknown anomaly when only the gate does.

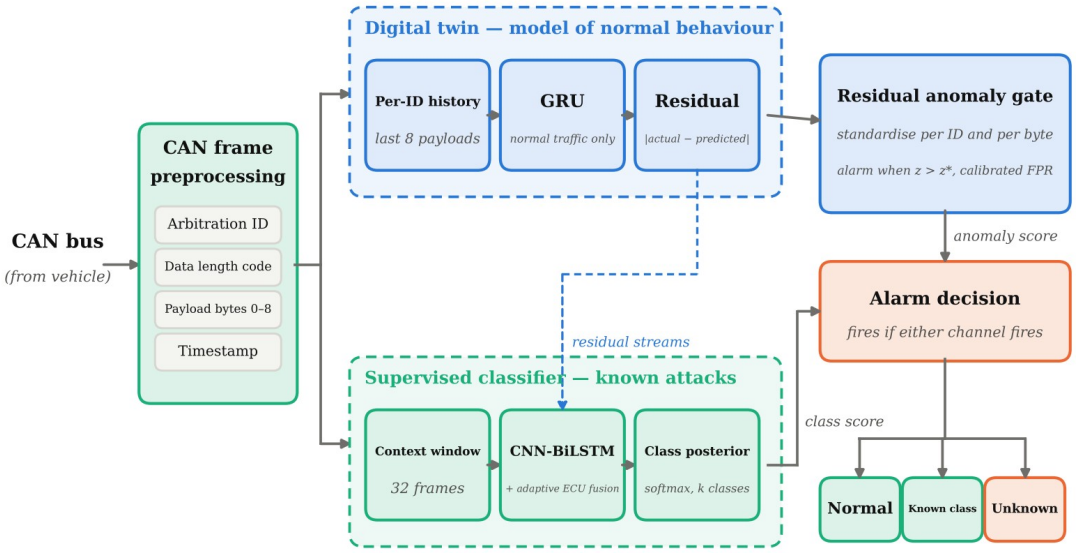


Figure 4.1: TwinShield-Fusion system architecture

The same system can be read a second way, as a stack of layers rather than as a flow of data. Figure 4.2 gives that view, and it is the one the source tree is organised around.

The system is organised into five layers, shown in Figure 4.2. Each layer depends only on the layer below it, so that a change to the interface cannot affect how a model is fitted, and a change to the splitting strategy propagates upward through a single documented interface rather than through scattered assumptions.

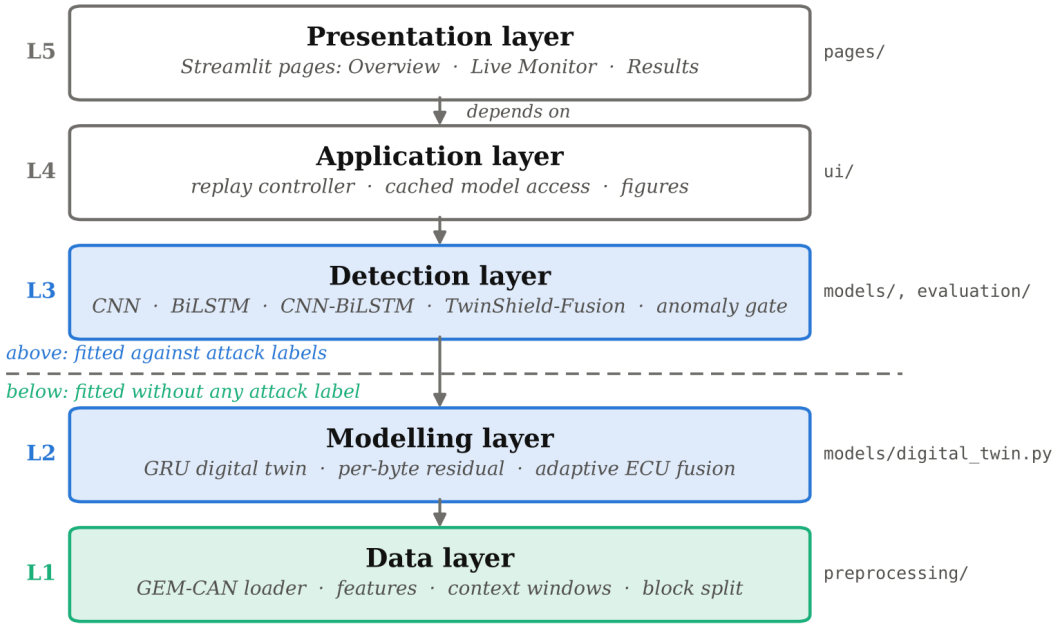


Figure 4.2: Layered view of the same system

Each layer is listed below with the responsibility it holds and the part of the source tree that implements it. The labels L1 to L5 in Figure 4.2 correspond to the five entries.

L1, the data layer. Reads the GEM-CAN capture through a schema-adaptive loader, derives the frame-level features of Section 5.3, gathers the 32-frame context windows of Section 5.4 and performs the block-wise split. Its output is cached, so the expensive preparation runs once and every later experiment reuses the identical split. Implemented in preprocessing/. This layer is the only one that touches the raw capture.

L2, the modelling layer. Fits the GRU digital twin on normal traffic, computes the per-byte residual of Equation 5.2 and pools residuals into per-ECU streams combined by additive attention. Implemented in models/digital_twin.py and models/residual_fusion.py. Nothing in this layer reads an attack label, which is the property the anomaly gate later depends on.

L3, the detection layer. Holds the three supervised baselines, the proposed TwinShield-Fusion detector and the residual anomaly gate, together with the metric code that scores them. Implemented in models/ and evaluation/. The detectors are fitted against labels; the gate is calibrated from normal traffic alone, and sits here because it issues decisions rather than because it is trained like its neighbours.

L4, the application layer. Provides the replay controller that advances the live demonstration frame by frame, and the cached model access that keeps the four detectors resident between interface interactions so that a replay tick costs a forward pass rather than a reload. Implemented in ui/.

L5, the presentation layer. Renders the three pages of the interface, Overview, Live Monitor and Results, and nothing else. It performs no computation of its own: every number it displays is returned by a lower layer. Implemented in pages/.

Each layer depends only on the layer beneath it, so a change to the interface cannot affect how a model is fitted, and a change to the splitting strategy propagates upward through a single documented interface rather than through scattered assumptions.

The separation between the modelling and detection layers is the architecturally significant one, and the dashed line in Figure 4.2 marks it. The digital twin and the residual belong to the modelling layer because they are fitted without reference to any attack label; the detectors belong to the detection layer because they are fitted against

labels. The anomaly gate sits in the detection layer but consumes only modelling-layer outputs, which is what allows it to be calibrated without attack data.

4.3 END-TO-END PIPELINE

Figure 4.3 shows the seven stages through which a frame passes, banded by the layer of Figure 4.2 that each stage belongs to, with the report section describing each stage shown in the corner of its box. Raw frames are converted to features, gathered into context windows, passed through the digital twin to produce a residual, pooled into per-ECU streams, and finally presented to both the supervised detector and the anomaly gate, whose outputs are combined into a single alarm decision.

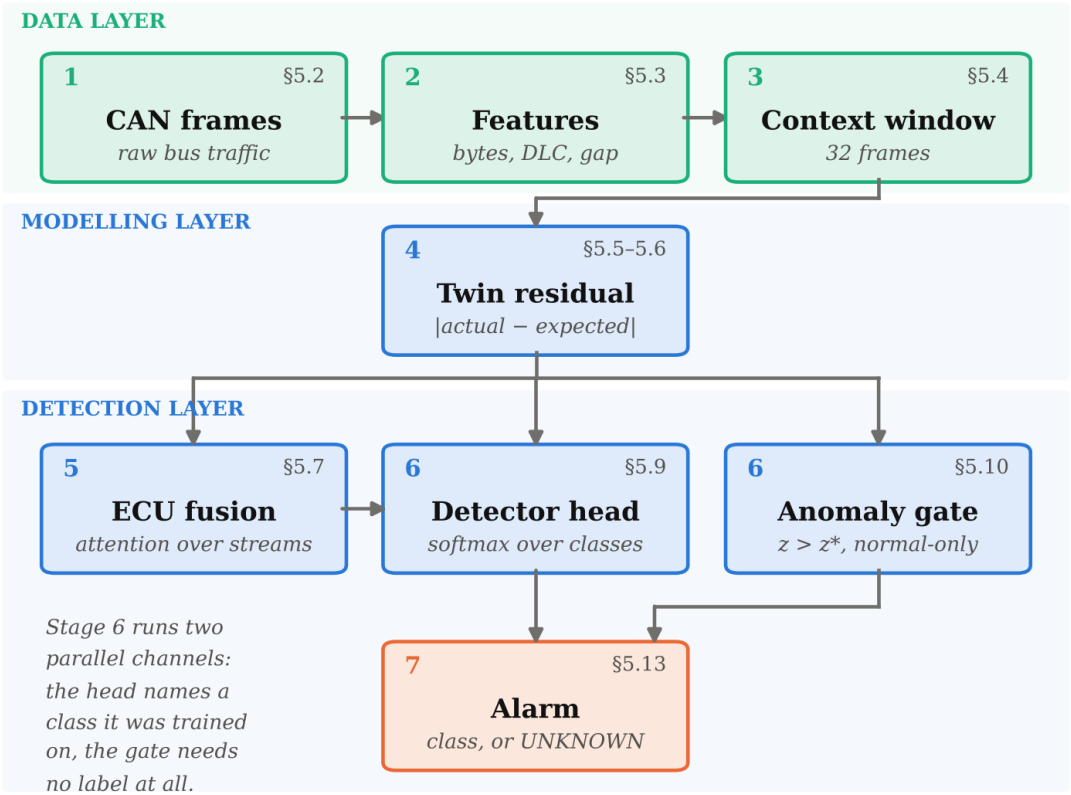


Figure 4.3: End-to-end processing pipeline: seven stages from raw frames to alarm

The colours carry the division of labour. Green marks the stages that handle raw data, blue the stages contributed by this work, and orange the alarm they produce. The digital-twin residual stage produces a quantity that is defined without reference to any attack label, and the combined detector-and-gate stage is where a decision can be reached either by recognition or by deviation.

4.4 USE CASE DESIGN

Figure 4.4 shows the use cases and the two actors. The evaluator drives the live comparison and reads its verdicts; the researcher additionally inspects the quantitative results and the attributions behind them. Neither actor trains a model from the interface, because training is performed offline by design.

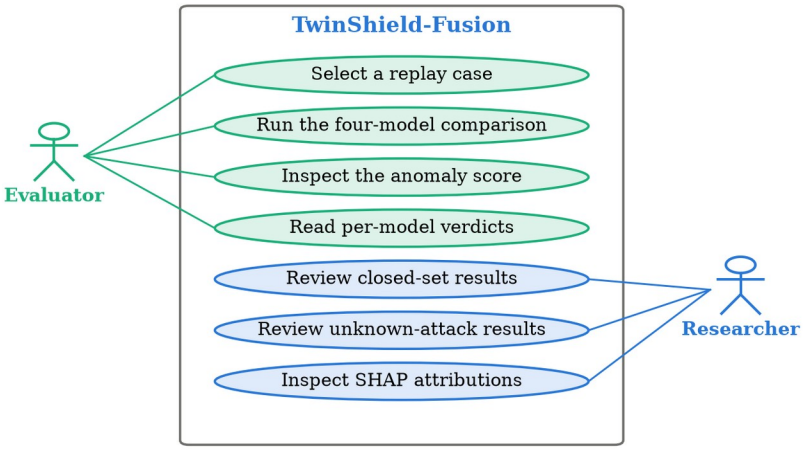


Figure 4.4: Use case diagram

4.5 DATA FLOW DESIGN

4.5.1 CONTEXT DIAGRAM (LEVEL 0)

At the highest level the system has one data source and one user, as shown in Figure 4.5. The GEM-CAN capture supplies CAN frames; the evaluator selects a replay case and receives alarms and anomaly scores.

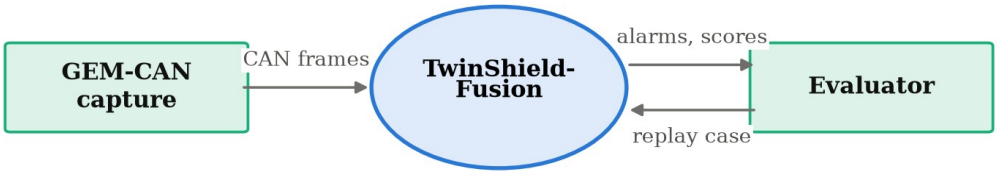


Figure 4.5: Level-0 data flow diagram (context diagram)

4.5.2 LEVEL-1 DATA FLOW DIAGRAM

Figure 4.6 decomposes the system into six processes and four data stores. Processes 1.0 and 2.0 prepare the data once and cache the result in store D2, so that repeated experiments do not repeat the preparation. Process 3.0 produces residuals from the digital twin, 4.0 pools them into ECU streams, 5.0 runs the four detectors and 6.0 applies the anomaly gate and reaches a decision. Store D4 holds the gate statistics, which are the only persistent state derived exclusively from normal traffic.

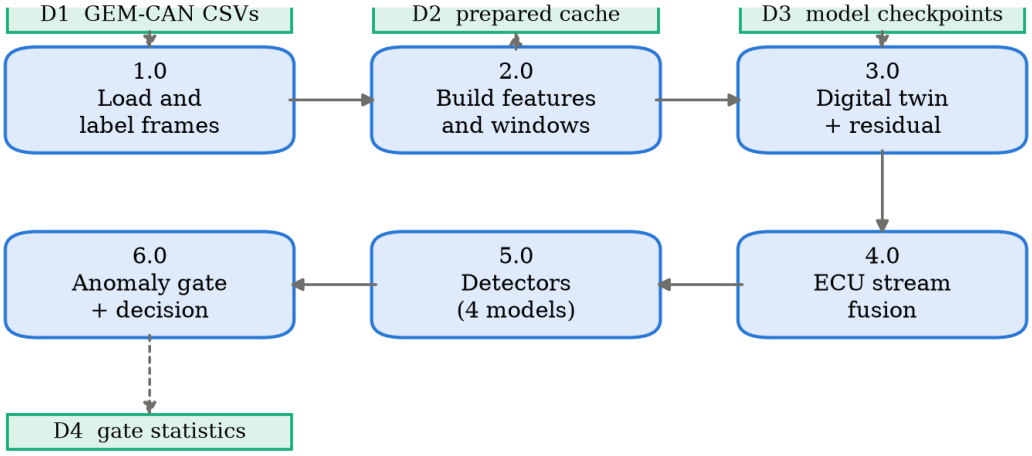


Figure 4.6: Level-1 data flow diagram

4.6 SEQUENCE OF A DETECTION REQUEST

Figure 4.7 traces a single replay tick. The evaluator presses the run control; the live monitor obtains the four detectors from the model cache, requests residuals for the newly revealed frames, scores them through the anomaly gate, runs four forward passes and returns verdicts, counters and trace data for rendering. Three frames are revealed per tick, and the entire sequence completes in approximately 12 ms.

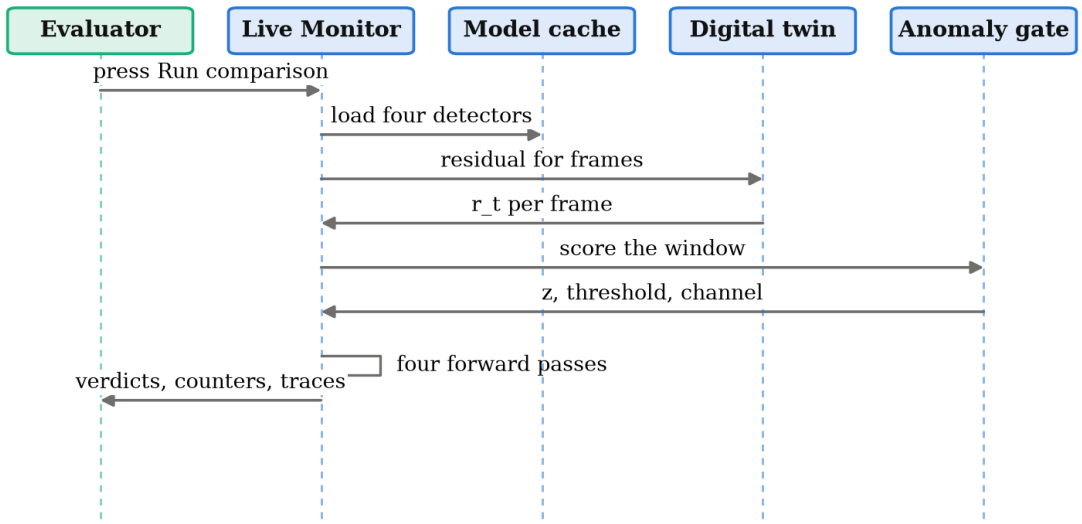


Figure 4.7: Sequence diagram of one replay tick

4.7 MODULE STRUCTURE

Table 4.1 lists the principal modules and their responsibilities. The structure mirrors the layering of Figure 4.2, so that a reader who has understood the architecture can locate any component in the source tree.

Table 4.1: Module structure

Module	Responsibility
preprocessing/loader.py	Schema-adaptive GEM-CAN loader; repairs the corrupted header and derives class labels
preprocessing/preprocessing.py	Frame-level features, with and without the arbitration identifier
preprocessing/windowing.py	Context windows, block-wise split, per-ECU history and residual streams
preprocessing/prepare.py	Cached end-to-end preparation
models/digital_twin.py	GRU digital twin and residual computation
models/residual_fusion.py	Adaptive multi-ECU attention and the TwinShield detector
models/anomaly_gate.py	Two-view residual anomaly gate; fitting, calibration and scoring
models/cnn.py, bilstm.py, cnn_bilstm.py	Baseline detectors
train.py	Closed-set training pipeline for all architectures and both feature variants
train_zeroday.py	Leave-one-attack-out experiment
evaluation/metrics.py, audit.py	Metric computation; leakage, shortcut and duplicate audit
explainability/shap_explainer.py	SHAP attributions over window and ECU-stream inputs
ui/	Theme, shared components and cached model access

Module	Responsibility
app.py, pages/	Streamlit entry point and the three pages

4.8 ARTIFACT DESIGN

Training produces a set of artifacts that the application consumes. Keeping these as explicit files rather than in-memory state is what allows the interface to start instantly and what makes a demonstration independent of a training run. Table 4.2 lists them.

Table 4.2: Artifacts produced by training

Artifact	Produced by	Contents
prepared_dataset.pkl	prepare.py	Cached features, windows, split assignment and identifier vocabulary
digital_twin.pt	train.py	GRU twin weights
residuals.npy	train.py	Per-frame, per-byte residual for the whole capture
{variant}__{arch}.pt	train.py	Detector weights for each architecture and feature variant
results.json	train.py	Closed-set metrics, per-class F1, ablation and configuration
zeroday__*.pt, .npy	train_zeroday.py	Refitted twin, residuals and the four quarantined detectors
zeroday__gate.npz	train_zeroday.py	Gate means, standard deviations and calibrated threshold
zero_day.json	train_zeroday.py	Per-model alarm rates, gate characterisation and split sizes
shap.npz	compute_shap.py	Feature and ECU-stream attributions

4.9 DEPLOYMENT DESIGN

The system is deployed as a single-machine application. Training scripts are run from the command line and write artifacts to disk; the Streamlit application reads those artifacts and serves an interface on a local port. There is no database and no network service beyond the local web server, which keeps the deployment surface small and makes the whole system reproducible from a source checkout and a dataset download. A readiness check script is run before any demonstration to confirm that every artifact

exists, that the quarantined class is absent from the trained label space and present in the test split, and that one real inference completes.

4.10 SUMMARY

This chapter presented the five-layer architecture, the seven-stage pipeline, the use cases, the data flow at two levels of detail, the sequence of a replay tick, the module structure and the artifacts that connect training to the interface. The design decision with the greatest consequence is the separation of the modelling layer, fitted without attack labels, from the detection layer, fitted against them: it is what makes an attack-agnostic decision channel possible at all. The next chapter describes how each component is implemented.

CHAPTER 5
METHODOLOGY AND IMPLEMENTATION

5.1 INTRODUCTION

This chapter describes how each component of TwinShield-Fusion is implemented. Sections 5.2 to 5.4 cover the dataset, the features derived from it and the splitting strategy. Sections 5.5 to 5.7 cover the digital twin, the residual and the adaptive fusion. Sections 5.8 and 5.9 describe the baseline detectors and the proposed model. Section 5.10 specifies the anomaly gate, and Section 5.11 the leave-one-attack-out protocol. Sections 5.12 and 5.13 cover explainability and the application.

5.2 DATASET: GEM-CAN

Table 5.1 gives the composition of the capture. The imbalance is severe: normal traffic outnumbers brake tampering by roughly 152 to 1, which is addressed during training by class-weighted cross-entropy.

Table 5.1: Composition of the GEM-CAN capture

Class	Frames	Share	Arbitration identifiers used
Normal	101,039	70.4%	13 distinct identifiers
DoS	41,080	28.6%	00000000 only
Brake tampering	663	0.5%	18FF00F9, shared with normal traffic
Steering tampering	662	0.5%	18FF70F9, shared with normal traffic
Total	143,444	100%	14 identifiers including the injected one

The two attack families differ in a way that governs the entire design. The denial-of-service attack floods the bus with arbitration identifier 00000000. Because CAN arbitration is won by the numerically lowest identifier, this identifier takes priority over every legitimate message on the vehicle, and genuine traffic is starved. It is also trivially separable: no legitimate ECU on this vehicle transmits that identifier, so the identifier alone identifies the attack.

Brake and steering tampering do not flood. They inject a small number of forged frames on identifiers 18FF00F9 and 18FF70F9, both of which carry genuine traffic throughout the capture. The identifier is therefore useless as a discriminator and detection must

come from the payload or from timing. For brake tampering the forged payload is far outside the normal range: bytes DATA_4 and DATA_5 move from a mean of roughly 0.5 to 58.8 and 77.5 respectively. For steering tampering the forged values sit inside the range that identifier normally carries, which is why that class behaves so differently in Chapter 6.

A further property shapes the experimental design. The two tampering attacks are injected as an interleaved pair roughly 0.56 seconds apart, so any context window wide enough to provide temporal context contains both classes. A single window-level label would therefore be ambiguous, and the system classifies each frame individually using the frames that precede it.

5.3 FRAME-LEVEL FEATURE ENGINEERING

Each frame is represented by the eight payload bytes scaled to the unit interval, the data length code scaled by eight, and the logarithm of the inter-frame interval on the bus. Timing spans microseconds to seconds, so the logarithmic compression keeps the feature on a scale comparable to the others.

Two feature variants are produced. The first adds the interval since the previous frame on the same identifier and a one-hot encoding of the identifier itself. The second omits both. Dropping the same-identifier interval alongside the one-hot encoding is deliberate and is a point that is easy to get wrong: grouping frames by identifier in order to compute that interval is itself identifier information, so a variant that kept it would not be identifier-free in any meaningful sense.

5.4 CONTEXT WINDOWS AND LEAKAGE-CONTROLLED SPLITTING

Each frame is classified from the 32 frames that precede it, gathered by index rather than materialised, so that the full window tensor is never held in memory for the whole capture. Where fewer than 32 preceding frames exist within a block, the earliest available frame is repeated.

Splitting required care. Consecutive CAN frames are near-duplicates, so a uniformly random frame-level split would place almost-identical frames in both training and test and inflate every metric. The capture is therefore cut into contiguous blocks of 256

frames, and whole blocks are assigned to training, validation and test, stratified by the rarest class each block contains. Because a frame's context window is drawn only from inside its own block, no frame is ever seen by two splits.

A purely chronological split was considered and rejected. The tampering attacks occupy the opening nine per cent of the attack capture, so a chronological cut leaves the test set with no tampering frames at all and renders the rarest and most interesting classes unmeasurable. The block-wise scheme preserves them while still preventing window-level leakage. Blocks adjacent in time remain correlated under this scheme, and that residual weakness is reported in Section 6.14 rather than claimed away.

The resulting split contains 100,352 training, 21,504 validation and 21,588 test frames. Test support is 15,063 normal, 6,288 DoS, 119 brake tampering and 118 steering tampering frames.

5.5 THE GRU DIGITAL TWIN

The digital twin learns what each ECU normally says. Given the eight most recent payloads observed on a particular arbitration identifier, it predicts the next payload on that identifier. Figure 5.1 shows the structure.

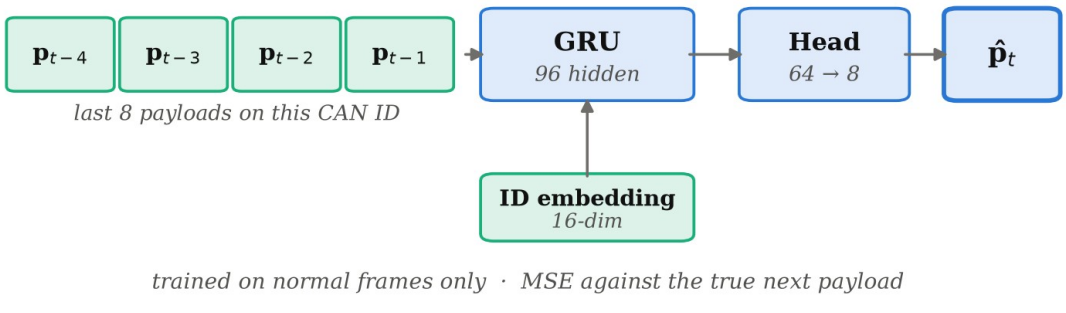


Figure 5.1: The GRU digital twin: predicting the next payload on an arbitration identifier

A 16-dimensional embedding of the identifier is concatenated to every step of the input sequence and again to the final hidden state, which allows a single network to serve every ECU on the bus rather than requiring one model per identifier. The recurrent core is a GRU with 96 hidden units; the head is a 64-unit rectified linear layer followed by an eight-output sigmoid, matching the scaled payload range.

Training minimises mean squared error against the true next payload, over normal frames from the training split only. This restriction is the single most important implementation detail in the project. The twin must not learn what attacks look like, because a model that had seen them would only be able to flag attacks it had already been shown, which is exactly the limitation the project exists to remove.

The per-ECU history used as input never crosses a block boundary, so the twin's context obeys the same split isolation as the classifier's context window.

5.6 RESIDUAL COMPUTATION

The residual is the element-wise absolute difference between the observed payload and the twin's prediction:

$$r_t = | p_t - \hat{p}_t | \qquad (5.1)$$

giving an eight-dimensional vector per frame, one component per payload byte. Under normal traffic the residual is small; when an attacker injects a forged payload on a legitimate identifier, the observed bytes stop matching the learnt pattern and the residual rises. This is the only mechanism in the system capable of flagging tampering on an identifier that also carries genuine traffic.

On held-out normal frames the twin achieves a mean absolute error of 10.52 bytes on the 0 to 255 scale. That is not a small error and it does not need to be. CAN payloads multiplex several independent signals, and some bytes are rolling counters or noisy sensor readings that are not predictable in principle. What detection requires is not a residual near zero, but a residual that differs reliably between normal and tampered traffic on the same identifier.

5.7 ADAPTIVE MULTI-ECU RESIDUAL FUSION

Each arbitration identifier is treated as a separate ECU stream. Within a context window, stream s is summarised by its mean residual, and streams absent from the window are masked. An additive attention layer then learns how much each stream should contribute to the decision for that particular window:

$$e_s = v^T \tanh(W h_s + b), \quad \alpha = \mathrm{softmax}(e) \qquad (5.2)$$

and the fused vector is the attention-weighted sum of the stream summaries, as shown in Figure 5.2. The intent is adaptivity: a flood concentrates evidence on one stream, while brake tampering concentrates it on the brake identifier's residual, and a fixed average over streams would dilute both.

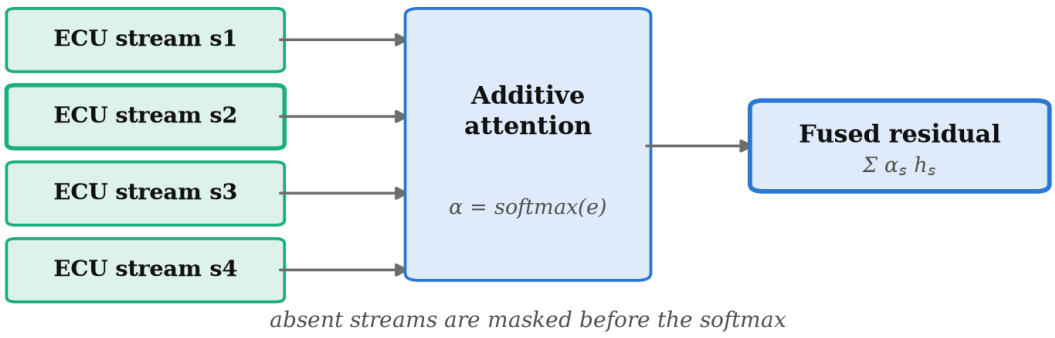


Figure 5.2: Adaptive multi-ECU residual fusion by additive attention

The attention weights are returned alongside the prediction so that the interface can display which ECU stream drove a given detection. Section 6.6 reports that on this dataset the attention layer contributes nothing measurable beyond the residual itself, and that result is presented as measured rather than omitted.

One subtlety applies to the identifier-free variant. The fusion branch normally keeps one fixed slot per identifier, which is harmless when the model is allowed to see identifiers. In the identifier-free experiment it would reintroduce exactly the information that variant exists to remove, because "slot one is active" would be equivalent to "identifier 00000000 is present". The streams are therefore rank-ordered by residual magnitude in that variant, so the slot index carries a rank rather than an identity.

5.8 BASELINE DETECTORS

Three supervised baselines are implemented, all consuming the raw context window and differing only in how they summarise it.

The CNN applies two one-dimensional convolutional layers with batch normalisation and rectified linear activation across the time axis, then concatenates mean and maximum pooling over time before a two-layer classification head. It captures local burst patterns but has no explicit model of order beyond its receptive field.

The BiLSTM runs a single bidirectional long short-term memory layer with 96 hidden units over the window and concatenates the final hidden state with the mean over time. It models longer-range order but has no convolutional front end, so local co-change between payload bytes must be learnt by the recurrence.

The CNN-BiLSTM combines both: convolutions extract local patterns, which the bidirectional recurrence then tracks across the window. This is the strongest baseline and the backbone that TwinShield-Fusion extends, so that the ablation in Section 6.6 compares like with like.

5.9 THE TWINSHIELD-FUSION DETECTOR

The proposed detector takes the CNN-BiLSTM backbone and extends it in two ways. Its input window is the raw feature vector concatenated with the eight-dimensional residual, so the backbone sees deviation alongside observation. Its classification head additionally receives the fused ECU residual vector from the attention layer of Section 5.7. Dropout is applied at 0.3 before each of the two head layers, and training uses class-weighted cross-entropy against the 152:1 imbalance.

This supervised component is, on its own, an ordinary closed-set classifier, and Chapter 6 shows that it behaves like one. The capability the project is concerned with comes from the gate described next.

5.10 THE RESIDUAL ANOMALY GATE

The gate is the component that does not depend on known attacks. It reads the same residual through two views, shown in Figure 5.3.

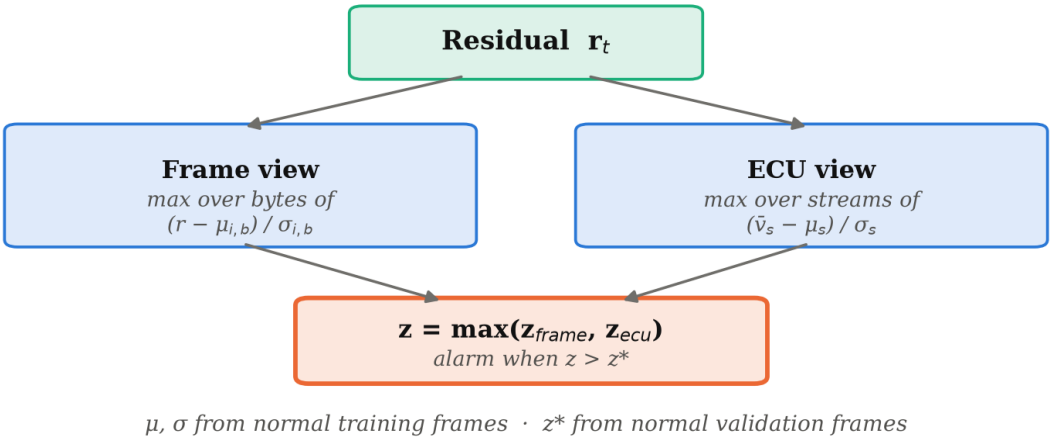


Figure 5.3: The two-view residual anomaly gate

The frame view asks how far a single frame departs from what its own ECU normally looks like, per payload byte:

$$z_frame(t) = \max \text{ over bytes } b \text{ of } (r[t,b] - \mu[i,b]) / \sigma[i,b] \qquad (5.3)$$

where i is the frame's arbitration identifier. This is the sharp channel: a forged payload that moves one byte far out of range shows up immediately and is not averaged away by the surrounding traffic. The ECU view asks how far a whole stream has drifted across the context window:

$$z_ecu(t) = \max \text{ over present streams } s \text{ of } (\bar{v}_s(t) - \mu_s) / \sigma_s \qquad (5.4)$$

and the gate fires when the larger of the two exceeds a threshold:

$$alarm(t) = 1 [\max(z_frame(t), z_ecu(t)) > z^*] \qquad (5.5)$$

The means and standard deviations are estimated on normal frames from the training split. The threshold is the $(1 - \alpha)$ quantile of the score over normal frames from the validation split, where α is the target false-positive rate, set to one per cent throughout. No attack frame of any class takes part in either step, and that is the whole basis for describing the gate as attack-agnostic: there is no signature in it that could have been memorised, only a model of normal.

Estimating the statistics on the twin's own training frames and reading the threshold off those same frames would understate the false-positive rate, because the twin predicts the frames it was fitted on better than any others. Calibrating on held-out normal frames against the same statistics is what makes the target rate mean something.

Standardising per identifier and per byte is not cosmetic. Some ECUs, and some bytes within an ECU's payload, sit at a high baseline residual because they carry counters or noisy readings, and a single global threshold fires on them continuously. Judging each byte of each ECU against its own normal behaviour is what allows one bus-wide threshold to be usable at all. A floor of one byte is applied to every standard deviation, so that a byte the twin happens to predict almost perfectly cannot turn ordinary jitter into an enormous score.

5.11 THE LEAVE-ONE-ATTACK-OUT PROTOCOL

One attack class is designated unknown and removed from every fitting stage. Deleting its label is not sufficient, and this is the point at which a naive quarantine leaks. A frame labelled normal whose 32-frame context window contains injected brake frames has still shown the detector what the attack looks like, and a normal frame whose per-ECU history contains them has still shown the digital twin. Four separate exclusions are therefore applied, as shown in Figure 5.4.

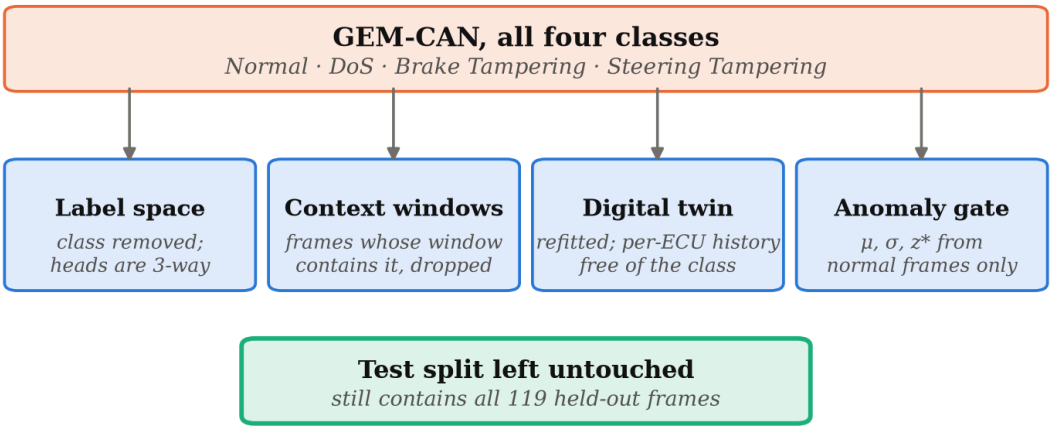


Figure 5.4: Leave-one-attack-out quarantine applied at four separate stages

1. The detector head is reduced to three classes, so the quarantined class has no output index at all and cannot be emitted even in principle.
2. Every training and validation frame whose context window contains a frame of the class is dropped from the fitting set.
3. The digital twin is refitted from random initialisation on normal frames whose per-ECU history is free of the class.

4. The gate's statistics and threshold are taken from normal frames only, as described in Section 5.10.

The test split is left entirely untouched and still contains all 119 brake tampering frames. Because one model then has a label the others lack, accuracy and macro-F1 are not comparable across models under this protocol. The comparison is therefore made on the question every deployed detector must answer, which is well posed in any label space: did you raise an alarm? The alarm rate is reported on each true class, which is detection rate on the attack classes and false-positive rate on the normal class.

5.12 EXPLAINABILITY WITH SHAP

Attributions are computed with a gradient-based SHAP explainer against the trained detector. The model takes two informative inputs, the raw-plus-residual context window and the per-ECU residual streams, and both are explained, so that an alert can be traced both to specific payload bytes and timing features and to the ECU whose residual mattered. Attributions are summed over the 32-frame window and averaged over a class-stratified sample of test frames. They are computed once after training and cached, so the interface loads them instantly.

5.13 THE APPLICATION

The system is delivered through a three-page interface. The overview page presents the pipeline, the dataset and the headline detection rates. The live monitor replays recorded frames through all four detectors simultaneously and shows a verdict card per model. The results page presents the quantitative evaluation, the digital twin, the closed-set comparison, the attributions and the dataset audit.

Two replay cases are offered. The first uses an attack class every model was trained on, and serves as a control: all four models should detect it, which demonstrates that the baselines are correctly trained and are not failing for some unrelated reason. The second uses the quarantined class. Each model card reports a catch count beside a false-alarm count, because a detector that alarmed on everything would otherwise appear to perform perfectly.

An implementation detail governs the honesty of the replay. The attack capture is already under attack from its first frame, so no stretch of it runs clean and then turns

malicious. Each replay case is therefore assembled from a lead-in taken from the normal-driving capture followed by a slice of the attack capture. The two recordings are joined, which the interface states openly rather than disguising. No frame is altered, reordered or generated, and each frame is still classified from its own true preceding context, because context windows never cross a block boundary.

5.14 SUMMARY

This chapter described the dataset and its two very different attack families, the feature variants and the block-wise split that controls leakage, the GRU digital twin and the residual it produces, the attention-based fusion over ECU streams, the three baselines and the proposed detector, the two-view anomaly gate and its normal-only calibration, the four-stage quarantine protocol, the attribution method and the interface. The next chapter reports what all of this measures.

CHAPTER 6
RESULTS AND DISCUSSION

6.1 INTRODUCTION

This chapter evaluates TwinShield-Fusion at each stage. Section 6.2 defines the metrics. Section 6.3 reports the dataset audit, which determines how every later number should be read. Section 6.4 reports the digital twin's prediction quality. Sections 6.5 and 6.6 give the closed-set comparison and the ablation. Sections 6.7 to 6.9 give the unknown-attack experiment, which is the central result. Sections 6.10 and 6.11 report the cost and the runtime, Section 6.12 the interface, and the chapter closes with discussion and limitations.

6.2 EVALUATION METRICS

For the closed-set experiments, performance is reported as macro-averaged F1 over the four classes. For a class c with true positives TP, false positives FP and false negatives FN:

$$Precision_c = TP_c / (TP_c + FP_c), \quad Recall_c = TP_c / (TP_c + FN_c) \quad (6.1)$$

$$F1_c = 2 \cdot Precision_c \cdot Recall_c / (Precision_c + Recall_c) \quad (6.2)$$

The macro average takes the unweighted mean over classes, so that brake tampering, with 119 test frames, counts as much as normal traffic, with 15,063. Accuracy is reported only where it is informative, for reasons Section 6.3 makes clear.

For the leave-one-attack-out experiment, macro-F1 is not defined across models with different label spaces. The metric there is the alarm rate on each true class: the fraction of that class's held-out test frames on which the model raised any alert. On the attack classes this is a detection rate; on the normal class it is a false-positive rate.

6.3 DATASET AUDIT AND THE ARBITRATION-ID SHORTCUT

Before training anything, the dataset was audited for shortcuts and leakage. The audit found a severe shortcut, and it changes how every closed-set number in this chapter should be interpreted.

Table 6.1: Dataset audit findings

Finding	Measured value	Consequence
Arbitration-ID lookup accuracy	98.9%	Accuracy is not a meaningful headline metric
Arbitration-ID lookup macro-F1	0.498	The shortcut solves DoS only, not tampering
Lookup F1 on DoS	1.00	DoS is separable without any learning
Lookup F1 on brake tampering	0.00	Tampering is where payload modelling must earn its place
Lookup F1 on steering tampering	0.00	As above
Byte-identical frames	87.4%	Any frame-level metric is inflated
Test frames with a twin in train	80.6%	Block splitting limits but does not remove this

A lookup table that predicts the majority class of each arbitration identifier, using no other information and performing no learning at all, reaches 98.9 per cent accuracy. Its per-class F1 is 1.00 on DoS and 0.00 on both tampering classes, giving a macro-F1 of 0.498. The reason is simple: every DoS frame carries identifier 00000000 and no normal frame does, so DoS is trivially separable, while the tampering classes reuse legitimate identifiers and cannot be separated by identifier at all.

Two consequences follow. Accuracy must not be used as the headline metric on this dataset, and the value of any payload-modelling component must be judged on the tampering classes. Every model in this project is therefore additionally trained with the arbitration identifier removed from the input, so that the shortcut can be eliminated rather than merely acknowledged.

6.4 DIGITAL TWIN PREDICTION QUALITY

On held-out normal frames the twin achieves a mean absolute error of 10.52 bytes on the 0 to 255 payload scale. Interpreted naively this looks poor, and it is worth stating plainly why it is not the relevant quantity. The twin is not a compression model and is not required to reconstruct payloads exactly. Several bytes in the capture are rolling counters or noisy sensor readings whose next value is not predictable from the preceding eight, and no model fitted on this input can drive their error to zero.

What detection requires is that the residual be reliably different between normal and tampered traffic on the same identifier. Section 6.8 shows that on one identifier it is, by

a factor of 8.9, and that on another it is not. Those two results, not the aggregate error, determine what the gate can do.

6.5 CLOSED-SET COMPARISON

Table 6.2 and Figure 6.1 report macro-F1 for every model on the shared held-out test split, with and without the arbitration identifier available as an input.

Table 6.2: Closed-set macro-F1 on the shared held-out test split

Model	CAN ID included	CAN ID excluded
CNN	0.9968	0.9914
BiLSTM	0.9948	0.9895
CNN-BiLSTM	0.9979	0.9914
GRU digital twin + CNN-BiLSTM	0.9989	0.9979
TwinShield-Fusion	0.9979	0.9979

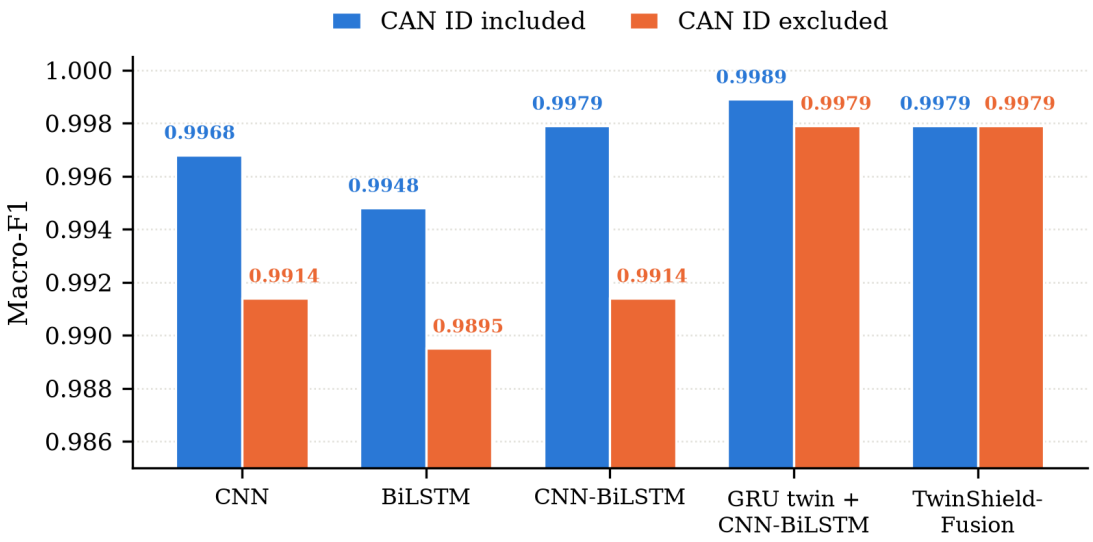


Figure 6.1: Closed-set macro-F1 for all five configurations

With the identifier included, the four architectures carried into the unknown-attack experiment—CNN, BiLSTM, CNN-BiLSTM and TwinShield-Fusion—span 0.0031 macro-F1; across all five rows of Table 6.2, which includes the intermediate GRU-twin configuration, the span is 0.0041. Neither figure is evidence that one architecture is better than another; it is evidence that the benchmark has stopped measuring architecture at all. Given that a lookup table on the identifier alone already scores 98.9

per cent accuracy, there is very little label information left for a sequence model to contribute, and every model is competing for the remainder.

Removing the identifier is more informative. There the digital-twin residual does measurable work, lifting the CNN-BiLSTM backbone from 0.9914 to 0.9979 macro-F1, a gain of 0.65 percentage points on a task where the shortcut has been eliminated. This is the closed-set result the project would defend; the identifier-included column is reported for completeness and because omitting it would be selective.

6.6 ABLATION STUDY

Table 6.3 and Figure 6.2 add one component at a time to the same CNN-BiLSTM backbone, so that each contribution can be isolated.

Table 6.3: Ablation on the CNN-BiLSTM backbone (macro-F1)

Configuration	CAN ID included	CAN ID excluded
CNN-BiLSTM backbone	0.9979	0.9914
+ digital twin (scalar residual)	0.9958	0.9965
+ residual vector (8 bytes)	0.9989	0.9979
+ adaptive multi-ECU fusion	0.9979	0.9979

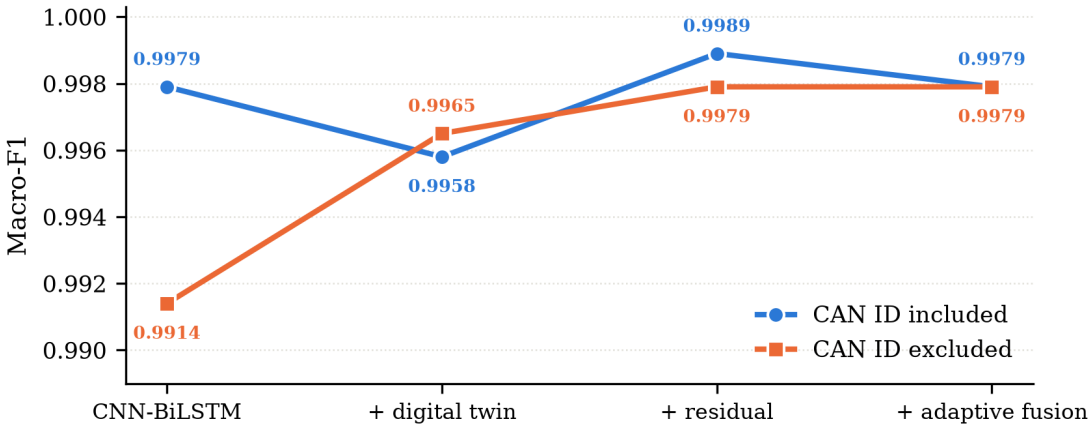


Figure 6.2: Ablation ladder with and without the arbitration identifier

On the identifier-free variant the ladder rises monotonically from 0.9914 to 0.9979 as the residual is introduced, which is the expected and desired pattern. One result must be stated plainly rather than glossed: adaptive fusion contributes nothing on top of the residual vector, moving macro-F1 from 0.9979 to 0.9979. The gain in this experiment comes from the residual, not from the attention layer.

This does not make the fusion branch useless, but it does bound the claim that can be made for it. Its attention weights remain interpretable and are displayed in the interface to show which ECU stream drove a detection, which is a usability benefit rather than an accuracy one. On a bus with more simultaneously attacked ECUs than this dataset contains, the branch might contribute measurably; on this dataset it does not, and the report says so.

6.7 DETECTION OF A QUARANTINED ATTACK

Table 6.4 and Figure 6.3 give the central result of the project. Brake tampering was removed from the label space, from every training context window, from the digital twin's per-ECU history and from the gate's calibration set, as described in Section 5.11. All four models were then evaluated on the untouched test split, which still contains its 119 frames.

Table 6.4: Leave-one-attack-out: brake tampering quarantined from every training stage

Model	Detected	False alarms	Known-class macro-F1
CNN	0.84%	0.03%	0.6667
BiLSTM	0.00%	0.08%	0.7607
CNN-BiLSTM	25.21%	0.31%	0.7787
TwinShield-Fusion	100.00%	2.26%	0.7829

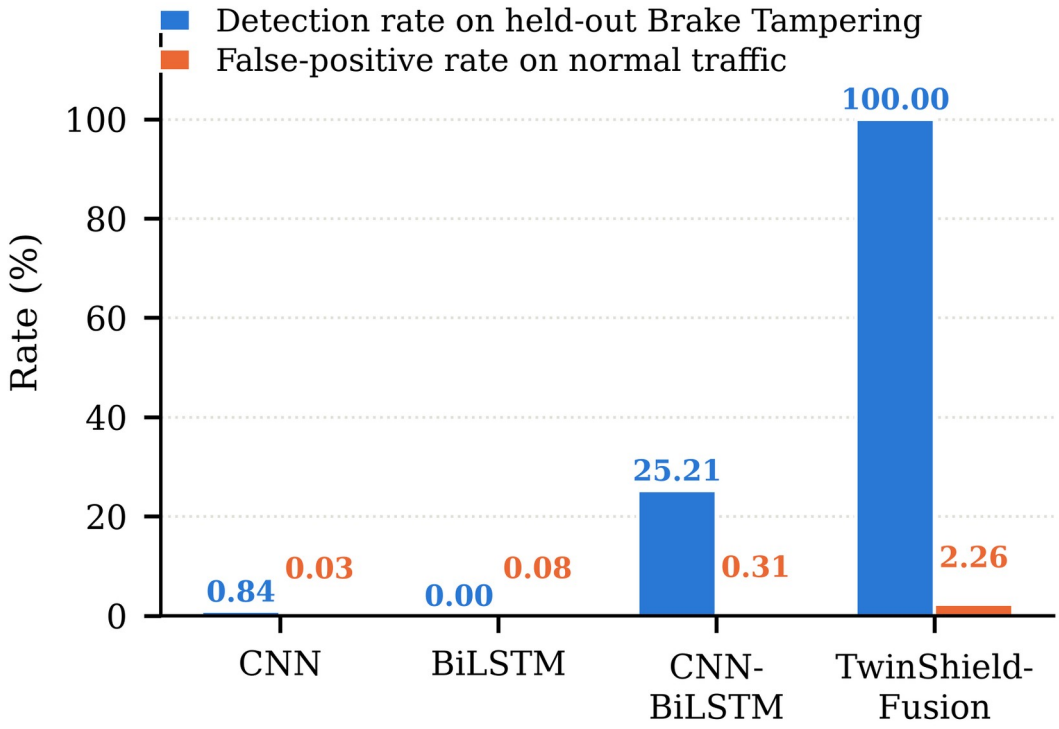


Figure 6.3: Detection of a quarantined attack class against the false-positive cost

The final column needs reading carefully, because it is lower than a reader might expect. It is macro-F1 over the three classes each model was still trained on, and macro-F1 weights those three equally. Every model classifies normal traffic and DoS almost perfectly, with per-class F1 between 0.991 and 0.9996. What drags the column down is steering tampering, where per-class F1 runs from 0.016 for the CNN to 0.354 for TwinShield-Fusion even though that class was present in training throughout. The column therefore makes a narrower point than it first appears to: the baselines are not undertrained on the easy classes, and their failure on the quarantined class is a property of the label space rather than of the optimiser; but low-rate payload tampering is evidently hard for a supervised detector even when it is labelled.

The three supervised baselines flag 0.84 per cent, 0.00 per cent and 25.21 per cent of the held-out frames. TwinShield-Fusion flags all 119.

The credit belongs to a specific component, and the decomposition is worth stating rather than implying. TwinShield-Fusion's supervised head contributes none of that detection, because it has no label to assign; the anomaly gate contributes all of it. The classifier is not what generalises here. The residual is. Claiming that the architecture generalises to unseen attacks would misattribute the result.

CNN-BiLSTM's 25.21 per cent deserves comment, because it is not recognition. That model has no output for brake tampering either. What it does is assign some of those frames to steering tampering, the other tampering class it was trained on, which shares an identifier family and a timing signature. The alarm happens to be correct, but for the wrong reason, and the two weaker baselines do not manage even that.

6.8 WHERE THE RESIDUAL WORKS, AND WHERE IT DOES NOT

The gate detects an attack when that attack pushes payload bytes outside what the twin has learnt is normal for the relevant identifier. This is a real condition rather than a formality, and Figure 6.4 shows it being met in one case and failing in the other.

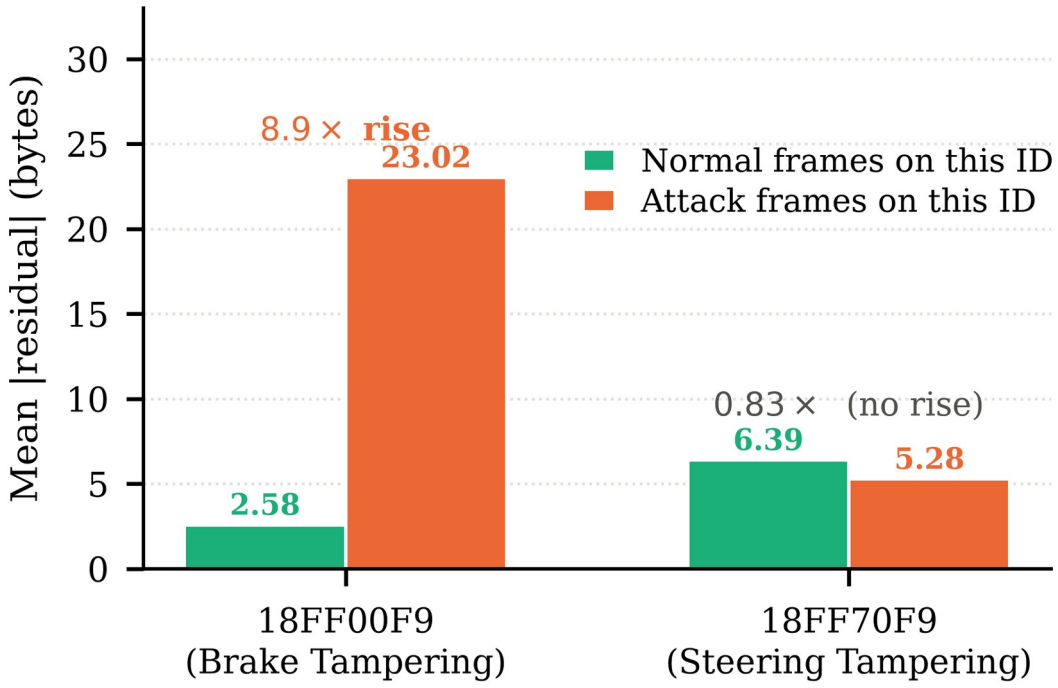


Figure 6.4: Mean digital-twin residual on the two tampering identifiers

On identifier 18FF00F9 the mean residual rises from 2.58 bytes on normal frames to 23.02 bytes under brake tampering, a factor of 8.9. The twin localises the attack precisely: bytes DATA_4 and DATA_5 carry almost all of the increase, which matches the forged values described in Section 5.2.

On identifier 18FF70F9 the mean residual under steering tampering is 5.28 bytes against 6.39 on normal traffic. It is lower than normal, not higher. The forged steering payloads fall inside the range the twin already accepts for that identifier, and a score

defined as a positive deviation above learnt-normal therefore cannot fire on them. This is the clearest limit of the current residual representation.

The boundary this places on the method is worth stating precisely, because it is easily overstated. Detection does not require prior knowledge of the attack, but it does require the attack to be anomalous in the payload. Those are different conditions, and conflating them would claim more than the measurements support.

6.9 CLOSED-SET AND OPEN-SET RESULTS COMPARED

Figure 6.5 places the two experiments side by side and summarises the argument of the whole project.

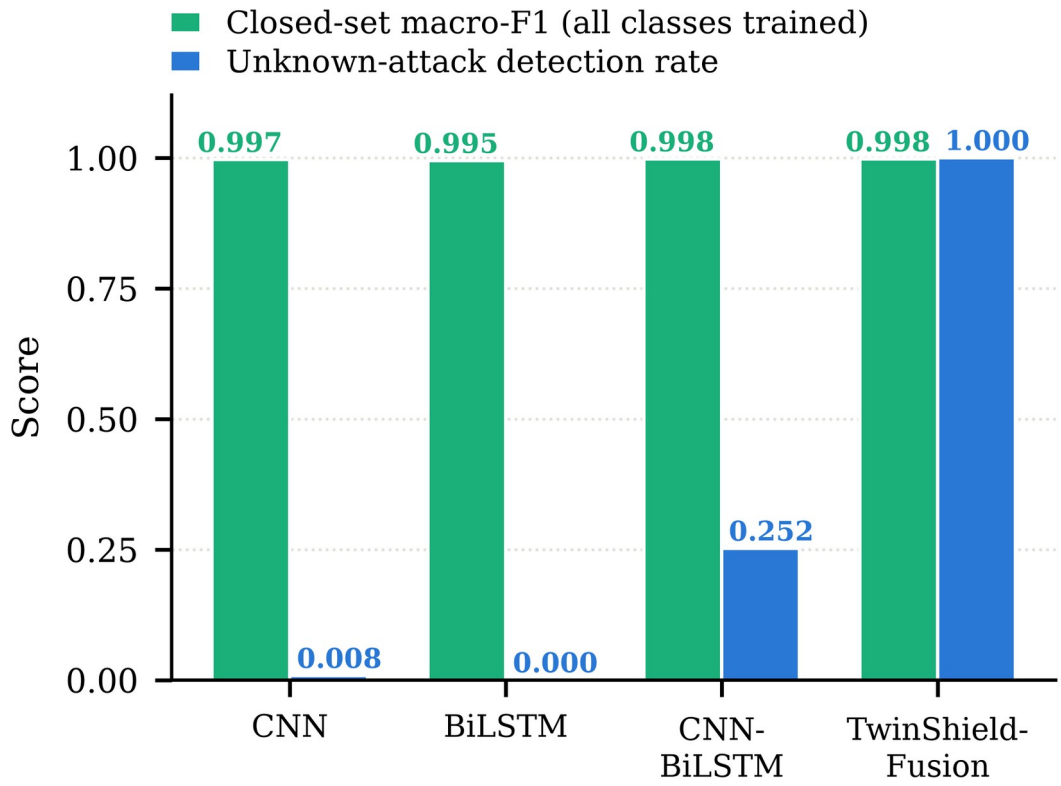


Figure 6.5: The same four models under two evaluation protocols

On the closed set the four models are indistinguishable, spanning 0.0031 macro-F1. On the quarantined class they are separated by more than 74 percentage points. The same models, the same data and the same training budget produce a benchmark that cannot tell them apart and a benchmark that separates them decisively. A closed-set score measures how well a model handles attacks somebody has already labelled, and nothing more.

6.10 THE COST OF THE SECOND CHANNEL

An always-on alarm detects every attack, so a detection rate reported without its false-positive rate is not a result at all. TwinShield-Fusion's false-positive rate on normal test traffic is 2.26 per cent, against 0.03 to 0.31 per cent for the baselines. On the 15,063 normal frames in the test split that is roughly 340 false alarms.

This is the price of the additional channel and it is not negligible. The gate was calibrated to a one per cent target; the realised rate is higher for two reasons. The supervised head contributes its own false alarms, which add to the gate's, and normal traffic in the validation and test splits is not identically distributed, so a quantile fitted on one does not transfer exactly to the other. A deployment that could not tolerate this rate would lower the target, at a cost in detection that the threshold sweep makes visible.

6.11 RUNTIME

A forward pass through all four models together with one gate evaluation takes 12.0 milliseconds per frame on CPU, approximately 3 milliseconds per model. The gate itself is two standardisations and a comparison, and its cost is negligible beside the twin's forward pass. The system has not been engineered for embedded deployment and no claim is made about line-rate operation on vehicle hardware, where a 250 kbit/s bus can deliver frames considerably faster than this.

6.12 USER INTERFACE

The interface delivers the comparison in a form an evaluator can run without reading the source. The live monitor shows four model cards side by side, each latching its verdict once the attack segment has played, with a catch count and a false-alarm count beneath. The anomaly score is plotted against its calibrated threshold, and the attention weights show which ECU stream drove the current decision.

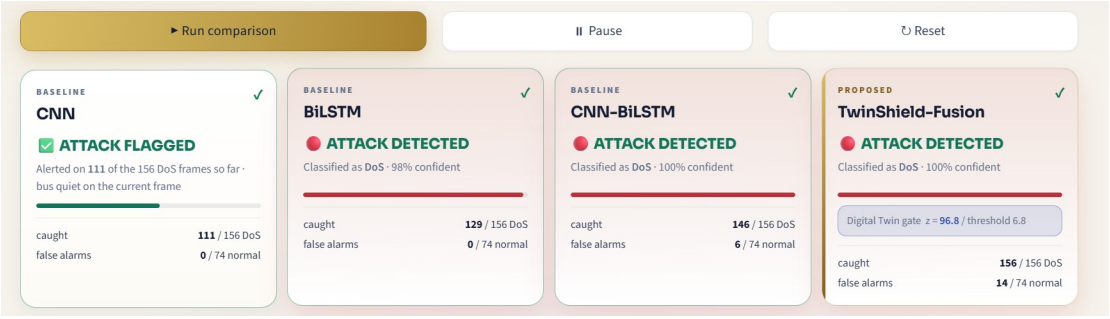


Figure 6.6: Live monitor on the known-attack control case

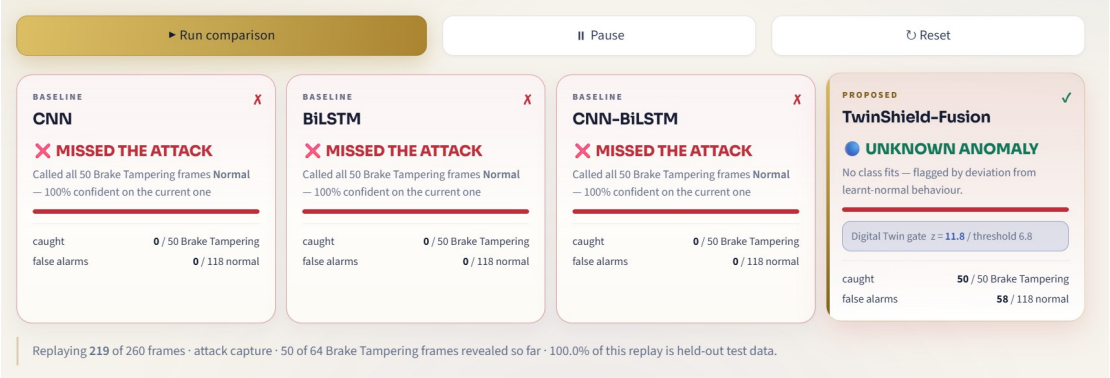


Figure 6.7: Live monitor on the quarantined attack case

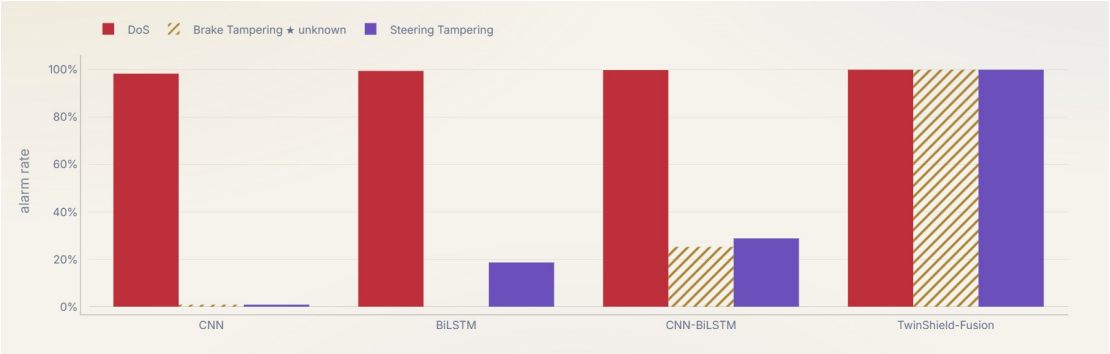


Figure 6.8: Results page: unknown-attack generalisation

6.13 CLASSIFICATION DIAGNOSTICS

Table 6.2 reports a single macro-F1 figure per model, which is a summary of the whole decision structure rather than a description of it. This section opens that summary up for the proposed detector, using the test-split probabilities produced by the same forward pass that produced Table 6.2. Three views are given: the confusion matrix, the receiver operating characteristic and the precision-recall curve.

[Insert figures/fig6_9_confusion.png — produced by report/make_eval_figs.py]

Figure 6.9: Confusion matrix for TwinShield-Fusion on the held-out test split

Figure 6.9 gives the full confusion structure behind the macro-F1 figure in Table 6.2. Each cell carries the frame count and, beneath it, that count as a share of the true class, so the rows sum to one hundred per cent. The diagonal dominance on normal traffic and on DoS is expected, since both are well represented in the split and DoS is separable by its arbitration identifier alone. The two tampering rows are the informative ones: with

119 and 118 test frames respectively, a handful of misclassified frames moves their per-class F1 noticeably, which is the reason macro-F1 rather than accuracy is used as the headline metric throughout this chapter.

[Insert figures/fig6_10_roc.png — produced by report/make_eval_figs.py]

Figure 6.10: One-vs-rest ROC curves with areas under the curve

Figure 6.10 plots the receiver operating characteristic for each class in turn, treating that class as positive and the remaining three as negative. Each panel shows the trade-off between true-positive rate and false-positive rate as the decision threshold on that class's probability is swept from one down to zero, and the dashed diagonal is the line a classifier with no discriminative power would follow. The area under each curve is reported in its panel title. The support is given at the lower right of each panel, because an area computed over 118 positive frames is a less stable estimate than one computed over 15,063, and the two should not be read with equal confidence.

[Insert figures/fig6_11_pr.png — produced by report/make_eval_figs.py]

Figure 6.11: One-vs-rest precision-recall curves with average precision

Figure 6.11 reports the same four decisions as precision against recall. On a split this imbalanced the precision-recall curve is the more honest of the two views, because the false-positive rate underlying a ROC curve is diluted by a large negative class whereas precision is not: a detector can hold a low false-positive rate and still raise mostly false alarms when the positive class is rare. The dashed line in each panel is the no-skill baseline, equal to that class's share of the test split, which is 0.698 for normal traffic and below 0.01 for each tampering class. The distance between the curve and its own baseline, rather than the absolute height of the curve, is what should be read.

These three views describe closed-set behaviour, where every class at test time was present during training. They say nothing about the quarantined class, for which no probability column exists; that case is the subject of Section 6.7, and it is measured by alarm rate rather than by any curve defined over a label space.

6.14 DISCUSSION

The results support a narrower claim than the headline figure might suggest, and stating that narrower claim correctly is the main intellectual content of this chapter.

What has been shown is that an unsupervised channel fitted on normal traffic alone can detect an attack class that was removed from every stage of training, where supervised classifiers of three different architectures cannot. What has not been shown is that the proposed architecture generalises; the ablation in Section 6.6 makes clear that the attention layer adds nothing here, and Section 6.7 makes clear that the supervised head contributes none of the unknown-attack detection. The mechanism that works is the digital-twin residual, and the architectural contribution is the arrangement that lets it operate alongside a classifier rather than instead of one.

The saturation reported in Section 6.5 deserves emphasis because it has a methodological moral. Four architectures spanning 0.0031 macro-F1 is a benchmark reporting that it has run out of discriminating power, not a field converging on a solution. Had this project reported only the closed-set table, it would have concluded that TwinShield-Fusion is comparable to a plain CNN, which is true and useless. The open-set protocol is what made the difference between the models visible.

6.15 LIMITATIONS

- **Single capture.** All results come from one vehicle on one closed track. Generalisation across attack classes has been shown; generalisation across vehicles or drive cycles has not, and would require a second dataset.
- **Payload-anomalous attacks only.** The gate cannot detect an attack whose forged values lie inside the learnt-normal range, as steering tampering demonstrates.
- **Known identifiers.** The quarantined attack arrives on an identifier the twin has a normal profile for. An attack on a never-observed identifier would fall back on the unknown-identifier embedding and has not been tested.
- **Replay attacks.** Retransmitting a genuine past payload produces a payload the twin finds predictable, so the residual stays low. Detecting replay requires timing or sequence modelling rather than payload modelling.
- **Residual correlation.** Blocks adjacent in time remain correlated, and 87.4 per cent of frames are byte-identical to another frame. Both inflate frame-level

metrics, though neither affects the leave-one-attack-out comparison asymmetrically, since all four models face identical conditions.

- **False-alarm rate.** At 2.26 per cent the proposed system raises an order of magnitude more false alarms than the baselines, which would matter in a deployment where alerts are acted upon manually.

6.16 SUMMARY

The audit established that accuracy is not usable on this dataset and that tampering is where payload modelling must earn its place. The closed-set benchmark was found to be saturated, the four compared architectures spanning 0.0031 macro-F1. Under a four-stage quarantine the same models separated by more than 74 percentage points, with TwinShield-Fusion detecting all 119 held-out frames of the removed class at a 2.26 per cent false-alarm rate. The mechanism was attributed to the digital-twin residual rather than to the architecture, and its boundary was measured on a second attack class where it does not work.

CHAPTER 7
TESTING AND VALIDATION

7.1 INTRODUCTION

This chapter describes how the system was tested. Section 7.2 gives the strategy, Section 7.3 the representative test cases and Section 7.4 the significant defects found during development, several of which changed the design.

7.2 TESTING STRATEGY

Table 7.1: Levels of testing

Level	Scope	Method
Unit	Loader, feature construction, windowing, gate arithmetic	Assertions on shapes, ranges and invariants
Integration	Training pipeline end to end on a reduced epoch budget	Short runs verifying artifacts are produced and loadable
System	All three interface pages rendered headlessly	Automated render check asserting no exception on any page
Readiness	Every artifact present and one real inference	check_demo.py, run before any demonstration
Protocol	Quarantine integrity	Assertion that the held-out class is absent from the label space and present in the test split

The protocol level deserves comment, because it tests a property rather than a behaviour. The entire argument of the project rests on the quarantined class being genuinely absent from training, so that property is asserted mechanically rather than trusted: the readiness check confirms that the trained models emit three classes, that the removed class is not among them, and that the test split still contains its frames.

7.3 TEST CASES

Table 7.2: Representative test cases

ID	Test	Expected	Result
T-01	Load the published CSVs without manual editing	143,444 frames, four classes	Pass
T-02	Every split contains every class	No empty class in train, val or test	Pass

ID	Test	Expected	Result
T-03	Residual array length matches frame count	143,444 rows	Pass
T-04	Detector probabilities sum to one	Within 1e-4 of 1.0	Pass
T-05	Attention weights normalised over present streams	Within 1e-3 of 1.0	Pass
T-06	Quarantined class absent from the label space	Three output classes, class not among them	Pass
T-07	Quarantined class present in the test split	119 frames	Pass
T-08	Gate calibration touches no attack frame	Fit and calibration indices are normal-only	Pass
T-09	Four forward passes within the tick budget	Under 100 ms for a tick	Pass, 12.0 ms
T-10	All three pages render without exception	No uncaught exception	Pass
T-11	Known-attack control detected by all four models	Four detections	Pass
T-12	Repeat run reproduces reported metrics	Identical to four decimal places	Pass

7.4 DEFECTS FOUND AND FIXED

Table 7.3 records the defects that materially changed the system. They are reported because each one produced output that looked plausible before it was found, and because the corrections are part of the contribution.

Table 7.3: Significant defects found during development

Defect	Symptom	Resolution
Partial quarantine	Held-out class removed from labels but still present in context windows and twin history	Added three further exclusions; contaminated windows dropped and the twin refitted
Slot-index leakage	Identifier-free fusion scored 0.9989, higher than expected	Fusion slots rank-ordered by residual magnitude so the index carries no identity
Same-ID interval leakage	Identifier-free variant retained a feature computed by grouping on identifier	Feature removed from that variant
Global anomaly threshold	Gate fired continuously on ECUs with noisy counters	Standardised per identifier and per byte, with a one-byte floor on sigma
Calibration on training frames	Realised false-positive rate far above the target	Threshold moved to held-out normal validation frames
Accuracy as headline	All models appeared near-perfect	Audit performed; macro-F1 and

Defect	Symptom	Resolution
metric		per-class F1 adopted, identifier-free variant added
Simulated figure	An early score-distribution figure used synthetic data	Figure removed; only measured values are plotted

7.5 SUMMARY

Testing covered five levels, from unit assertions to a protocol-integrity check that mechanically verifies the quarantine on which the project's central claim depends. Twelve representative test cases pass. Seven significant defects were found and fixed, and the most consequential of them, partial quarantine and slot-index leakage, would each have produced an inflated result had they gone unnoticed.

CHAPTER 8
CONCLUSION AND FUTURE WORK

8.1 CONCLUSION

This project set out to address a gap in learning-based CAN intrusion detection: the dominant evaluation protocol is closed-set, and a closed-set protocol cannot distinguish a detector that generalises from one that has memorised a catalogue of attacks.

The closed-set benchmark on GEM-CAN was found to be saturated. The four architectures compared, from a plain CNN to the proposed fusion model, span 0.0031 macro-F1, and a lookup table on the arbitration identifier alone reaches 98.9 per cent accuracy. Reporting only that table would have supported the conclusion that the proposed method is comparable to a plain convolutional network, which is accurate and entirely uninformative.

Under a leave-one-attack-out protocol that removes an attack class from the label space, from every training context window, from the digital twin's per-ECU history and from the anomaly threshold's calibration set, the same four models separate by more than 74 percentage points. TwinShield-Fusion detects all 119 held-out frames of the quarantined class at a 2.26 per cent false-positive rate, while the best supervised baseline reaches 25.21 per cent and the weakest reaches zero.

The mechanism responsible has been identified rather than assumed. The supervised head contributes none of that detection and the anomaly gate contributes all of it, so the finding is that the digital-twin residual generalises, not that the architecture does. The boundary of the mechanism has been measured on a second attack class: steering tampering leaves the residual below its normal level and is not detected. Detection therefore does not require prior knowledge of an attack, but it does require the attack to be anomalous in the payload.

8.2 FUTURE WORK

1. Validate across vehicles and drive cycles. A single capture cannot evidence generalisation beyond itself. Repeating the protocol on a second dataset from a different vehicle would test whether the gate's calibration transfers.

2. Strengthen the residual representation for payload-conservative attacks. Steering tampering is invisible to a magnitude-based score. A representation sensitive to joint structure across bytes, or to the temporal shape of a signal rather than its level, is the clearest direction for extending coverage.
3. Model timing explicitly for replay detection. A replayed genuine payload is predictable by construction, so the payload channel cannot see it. An inter-arrival model per identifier would complement the residual without duplicating it.
4. Reduce the false-alarm rate. At 2.26 per cent the second channel is expensive. Hysteresis over consecutive frames, or requiring agreement between the frame and ECU views rather than taking their maximum, would both trade detection for precision in ways worth measuring.
5. Evaluate against identifiers never seen in training, which is the weaker case the present evaluation does not cover.
6. Assess embedded feasibility. The current 3 milliseconds per model on CPU is far from line rate on a loaded bus, and quantisation or distillation of the twin would be the first thing to try.

8.3 SUMMARY

The project delivered a CAN intrusion detection framework combining a supervised detector with a digital-twin residual gate calibrated on normal traffic alone, a four-stage quarantine protocol for evaluating unknown-attack detection, a measured demonstration that the closed-set benchmark on this dataset is saturated, and a measured boundary on where the mechanism works. Six directions for future work have been identified, of which strengthening the residual representation for payload-conservative attacks is the most immediately valuable.

REFERENCES

[1] Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. ACM Computing Surveys, 41(3), Article 15. https://doi.org/10.1145/1541880.1541882

[2] Checkoway, S., McCoy, D., Kantor, B., Anderson, D., Shacham, H., Savage, S., Koscher, K., Czeskis, A., Roesner, F., & Kohno, T. (2011). Comprehensive experimental analyses of automotive attack surfaces. In Proceedings of the 20th USENIX Security Symposium. USENIX Association.

[3] Cho, K.-T., & Shin, K. G. (2016). Fingerprinting electronic control units for vehicle intrusion detection. In Proceedings of the 25th USENIX Security Symposium (pp. 911-927). USENIX Association.

[4] Cho, K., van Merrienboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. In Proceedings of EMNLP (pp. 1724-1734). ACL.

[5] Geng, C., Huang, S.-J., & Chen, S. (2021). Recent advances in open set recognition: A survey. IEEE Transactions on Pattern Analysis and Machine Intelligence, 43(10), 3614-3631. https://doi.org/10.1109/TPAMI.2020.2981604

[6] Hanselmann, M., Strauss, T., Dormann, K., & Ulmer, H. (2020). CANet: An unsupervised intrusion detection system for high dimensional CAN bus data. IEEE Access, 8, 58194-58205. https://doi.org/10.1109/ACCESS.2020.2982544

[7] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735-1780. https://doi.org/10.1162/neco.1997.9.8.1735

[8] International Organization for Standardization. (2015). Road vehicles - Controller area network (CAN) - Part 1: Data link layer and physical signalling (ISO 11898-1:2015).

[9] Kang, M.-J., & Kang, J.-W. (2016). Intrusion detection system using deep neural network for in-vehicle network security. PLoS ONE, 11(6), Article e0155781. https://doi.org/10.1371/journal.pone.0155781

[10] Koscher, K., Czeskis, A., Roesner, F., Patel, S., Kohno, T., Checkoway, S., McCoy, D., Kantor, B., Anderson, D., Shacham, H., & Savage, S. (2010). Experimental security analysis of a modern automobile. In Proceedings of the IEEE Symposium on Security and Privacy (pp. 447-462). IEEE.

[11] Lokman, S. F., Othman, A. T., & Abu-Bakar, M. H. (2019). Intrusion detection system for automotive controller area network (CAN) bus system: A review. EURASIP Journal on Wireless Communications and Networking, 2019, Article 184. https://doi.org/10.1186/s13638-019-1484-3

[12] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In Advances in Neural Information Processing Systems 30 (pp. 4765-4774).

[13] Miller, C., & Valasek, C. (2015). Remote exploitation of an unaltered passenger vehicle [Technical report]. Black Hat USA.

[14] Muter, M., & Asaj, N. (2011). Entropy-based anomaly detection for in-vehicle networks. In Proceedings of the IEEE Intelligent Vehicles Symposium (pp. 1110-1115). IEEE.

[15] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In Advances in Neural Information Processing Systems 32 (pp. 8024-8035).

[16] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

[17] Seo, E., Song, H. M., & Kim, H. K. (2018). GIDS: GAN based intrusion detection system for in-vehicle network. In Proceedings of the 16th Annual Conference on Privacy, Security and Trust (PST) (pp. 1-6). IEEE.

[18] Song, H. M., Woo, J., & Kim, H. K. (2020). In-vehicle network intrusion detection using deep convolutional neural network. Vehicular Communications, 21, Article 100198. https://doi.org/10.1016/j.vehcom.2019.100198

[19] Tao, F., Zhang, H., Liu, A., & Nee, A. Y. C. (2019). Digital twin in industry: State-of-the-art. IEEE Transactions on Industrial Informatics, 15(4), 2405-2415. https://doi.org/10.1109/TII.2018.2873186

[20] Taylor, A., Leblanc, S., & Japkowicz, N. (2016). Anomaly detection in automobile control network data with long short-term memory networks. In Proceedings of the IEEE International Conference on Data Science and Advanced Analytics (DSAA) (pp. 130-139). IEEE.

APPENDIX 1

ALGORITHMS

A1.1 DIGITAL TWIN TRAINING

```
Input:  normal training frames N, per-ECU history index H, identifier
index I
Output: fitted twin M

initialise M with random weights
for epoch = 1 to E do
    shuffle N
    for each minibatch B drawn from N do
        X  <- payload[ H[B] ]          // last 8 payloads on this
identifier
        id <- I[B]                     // identifier index for the
embedding
        Y  <- payload[B]               // the true next payload
        loss <- MSE( M(X, id), Y )
        update M by Adam on loss
    end for
end for
return M
```

A1.2 RESIDUAL ANOMALY GATE: FITTING AND CALIBRATION

```
Input:  residual r, identifier index I, ECU streams S, mask K,
        normal training indices F, normal validation indices C,
        target false-positive rate alpha
Output: gate (mu_frame, sigma_frame, mu_ecu, sigma_ecu, z*)

// frame view: per identifier, per payload byte
for each identifier i do
    mu_frame[i]    <- mean   of r[F where I = i]
    sigma_frame[i] <- stdev  of r[F where I = i]
    sigma_frame[i] <- max( sigma_frame[i], 1/255 )     // one-byte floor
end for

// ECU view: per stream, over windows in which the stream is present
for each stream s do
    v        <- mean over bytes of S[F, s]
    mu_ecu[s]    <- masked mean  of v where K[F, s] = 1
    sigma_ecu[s] <- masked stdev of v where K[F, s] = 1
    sigma_ecu[s] <- max( sigma_ecu[s], 1/255 )
end for

// calibration on held-out normal frames only
scores <- score( r[C], I[C], S[C], K[C] )
z*     <- quantile( scores, 1 - alpha )
return gate
```

A1.3 SCORING A FRAME

```
Input:  frame t, gate
Output: score z, firing channel

i  <- identifier of frame t
zf <- max over bytes b of ( r[t,b] - mu_frame[i,b] ) / sigma_frame[i,b]
```

```
ze <- -infinity
for each stream s present in the window of t do
    v  <- mean over bytes of S[t, s]
    ze <- max( ze, ( v - mu_ecu[s] ) / sigma_ecu[s] )
end for
if no stream is present then ze <- -infinity

z       <- max( zf, ze )
channel <- "frame" if zf >= ze else "ecu"
alarm   <- z > z*
return z, channel, alarm
```

A1.4 LEAVE-ONE-ATTACK-OUT QUARANTINE

```
Input:  dataset D, context windows W, per-ECU history H, unknown class u
Output: fitting sets free of class u

is_unknown   <- ( label = u )
window_clean <- NOT any( is_unknown[ W[t] ] )   for every frame t
hist_clean   <- NOT any( is_unknown[ H[t] ] ) AND NOT is_unknown[t]

train_idx <- { t : split[t] = train AND label[t] != u AND window_clean[t]
}
val_idx   <- { t : split[t] = val   AND label[t] != u AND window_clean[t]
}
test_idx  <- { t : split[t] = test }              // untouched

twin_fit  <- { t : split[t] = train AND label[t] = normal AND
hist_clean[t] }
gate_fit  <- { t : split[t] = train AND label[t] = normal AND
window_clean[t] }
gate_cal  <- { t : split[t] = val   AND label[t] = normal AND
window_clean[t] }

remap labels to a (k-1)-way space that does not contain u
return all index sets
```

APPENDIX 2

INSTALLATION AND RUNNING

A2.1 ENVIRONMENT SETUP

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Obtain the dataset
#    Download GEM_CAN_Dataset.zip from
https://zenodo.org/records/17834776
#    and extract so that the CSVs sit directly in:
#       data/GEM_CAN_Dataset/GEM_Normal_Driving_Raw.csv
#       data/GEM_CAN_Dataset/GEM_Attack_Scenario_Raw.csv
#       data/GEM_CAN_Dataset/GEM_Attack_Scenario_Raw_Labeled.csv
#
#    Or point elsewhere:   set GEMCAN_DIR=D:\path\to\GEM_CAN_Dataset
```

A2.2 RUNNING THE SYSTEM

```
python train.py            # closed-set experiment, both feature variants
python train_zeroday.py    # leave-one-attack-out, brake tampering held
out
python compute_shap.py     # SHAP attributions
python check_demo.py       # verify every artifact and run one real
inference
streamlit run app.py       # launch the interface

# Useful variants
python train.py --epochs 3                              # quick rehearsal
run
python train.py --only twinshield                       # retrain one
architecture
python train_zeroday.py --unknown "Steering Tampering"  # the negative
case
python train_zeroday.py --target-fpr 0.005              # tighter alarm
threshold
```

A2.3 COMMAND-LINE OPTIONS

Table A2.1: Principal command-line options

Script	Option	Default	Effect
train.py	--epochs	6	Detector training epochs
train.py	--dt-epochs	5	Digital twin training epochs
train.py	--variants	both	Feature variants to train (with_id, no_id)
train.py	--only	all	Retrain one architecture and merge the rest
train_zeroday.py	--unknown	Brake Tampering	Attack class to quarantine
train_zeroday.py	--target-fpr	0.01	Target false-positive rate for the gate threshold

Script	Option	Default	Effect
train_zeroday.py	--epochs	6	Detector training epochs
all	--seed	42	Random seed for reproducibility

APPENDIX 3

ARTIFACT AND PARAMETER REFERENCE

A3.1 MODEL HYPERPARAMETERS

Table A3.1: Model hyperparameters

Component	Parameter	Value
Context window	Frames of history	32
Digital twin	Identifier history length	8 payloads
Digital twin	GRU hidden units	96
Digital twin	Identifier embedding	16 dimensions
Digital twin	Head	64-unit ReLU, 8 outputs, sigmoid
Digital twin	Loss	Mean squared error
CNN baseline	Convolution channels	64, then 128
CNN baseline	Kernel sizes	5, then 3
BiLSTM baseline	Hidden units	96, bidirectional
CNN-BiLSTM	Convolution channels	64, 64
CNN-BiLSTM	LSTM hidden units	96, bidirectional
Fusion attention	Projection / output dimension	48 / 48
All detectors	Dropout	0.3
All detectors	Optimiser	Adam, learning rate 1e-3
All detectors	Batch size	512
All detectors	Loss	Class-weighted cross-entropy
Anomaly gate	Target false-positive rate	1%
Anomaly gate	Sigma floor	1 byte (1/255 on the scaled range)
Splitting	Block size	256 frames
Splitting	Train / val / test ratio	0.70 / 0.15 / 0.15
All experiments	Random seed	42

A3.2 SPLIT COMPOSITION

Table A3.2: Class support in each split

Split	Frames	Normal	DoS	Brake	Steering
Train	100,352	70,510	28,676	463	462
Validation	21,504	15,466	6,116	81	82

Split	Frames	Normal	DoS	Brake	Steering
Test	21,588	15,063	6,288	119	118
Total	143,444	101,039	41,080	663	662

Train and validation class counts are derived by subtraction from the published totals and the measured test support, and should be confirmed against the split report printed by the readiness check before the report is submitted.